MLP Classification Analysis Report

Introduction 3
Methodology 7
Results & Discussion (Parametric Study) 11
Conclusion 43

Introduction

Perceptron (single layer perceptron)

The Perceptron was the first significant algorithm that played a key role in the early development of Artificial Intelligence and Machine Learning. Developed in the 1950s by psychologist Frank Rosenblatt, this model was inspired by neurobiology. As mathematical theories matured and computing power increased, research in this area evolved from simple biological mimicry into a sophisticated field of computational intelligence.

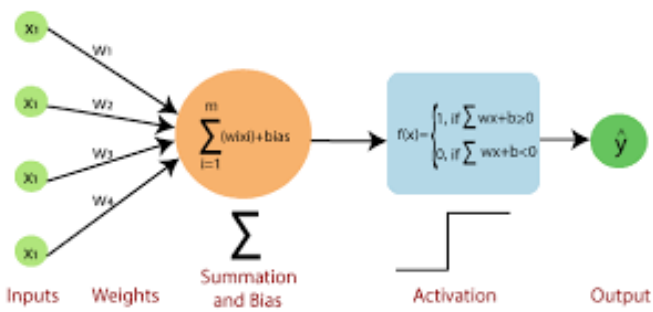


Figure 1: a Single-layer model

perceptron training algorithm

The Delta Rule is commonly used to train a single-layer perceptron. In this algorithm, the weights are updated based on the error between the target and the actual output. The update rule is expressed as:

$$Wi(n+1) = Wi(n) + \eta\delta x_i$$

η: learning rate δ:target-actual output xi: input.

Linear Separability and the XOR Problem:

A major limitation of the single-layer perceptron is its inability to handle problems that are not linearly separable. In other words, if the data cannot be divided into two classes using a single straight line (hyperplane), the perceptron fails to classify it correctly. The XOR problem is the classic example of this limitation, which eventually led to the development of more complex architectures.

Multi-layer Perceptron

To address the limitations of linear separability, the Multi-Layer Perceptron (MLP) was introduced. An MLP consists of fully connected "dense" layers that can model complex, non-linear relationships. An MLP architecture typically includes:

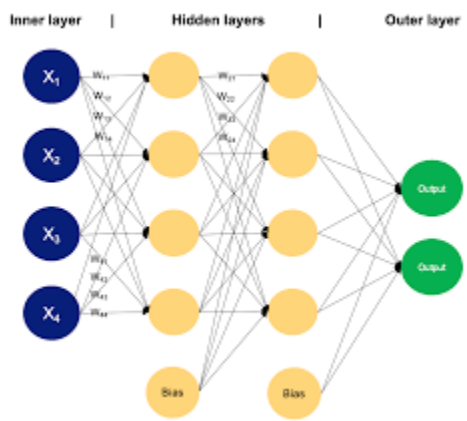


Figure 2.Multi-layer Perceptron

- **Input Layer**: Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers**: MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process information received from the input layer using weights and biases. They are responsible for extracting complex patterns from the data.
- **Output Layer**: This layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.

In an MLP, every node in one layer connects to every node in the next layer, allowing the network to transform data dimensions and learn non-linear decision boundaries.

Activation Function

To introduce non-linearity into the network, the weighted sum of inputs is passed through an activation function. Common functions include:

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$

- **ReLU (Rectified Linear Unit)**: $f(z) = \max(0, z)$
- **Tanh (Hyperbolic Tangent)**: $\tanh(z) = \frac{2}{1+e^{-2z}} - 1$

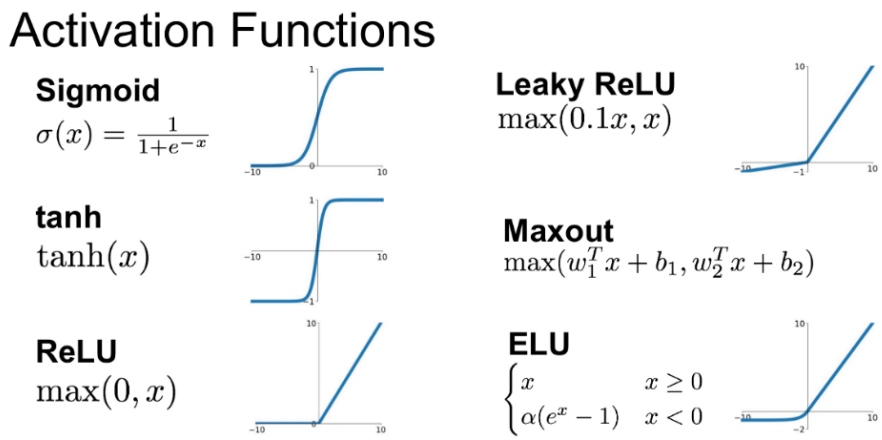


Figure 3.Activation Function

Loss Function

The performance of the network is measured using a loss function. In this experiment, for binary classification, I use **Binary Cross-Entropy**:

$$J = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Where:

- y_i is the actual label.
- $\hat{y}_i$ is the predicted label.
- N is the number of samples.

For regression problems the mean squared error (MSE) is often used:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Backpropagation

The primary goal of training an MLP is to minimize the loss function by iteratively adjusting the network's weights and biases. This is achieved through the Backpropagation algorithm, which follows three main steps:

1. **Gradient Calculation**: The gradients of the loss function with respect to each weight and bias are calculated using the chain rule of calculus.
2. **Error Propagation**: The error is propagated back through the network, layer by layer.
3. **Gradient Descent**: The network updates the weights and biases by moving in the opposite direction of the gradient to reduce the loss: $w_{new} = w_{old} - \eta \cdot \frac{\partial L}{\partial w}$

Where:

- w is the weight.
- η is the learning rate.
- $\frac{\partial L}{\partial w}$ is the gradient of the loss function .

Optimization

MLPs rely on optimization algorithms to iteratively refine the weights and biases during training. Popular optimization methods include:

- **Stochastic Gradient Descent (SGD)**: Updates the weights based on a small batch of data: $w = w - \eta \cdot \frac{\partial L}{\partial w}$
- **Adam Optimizer**: An extension of SGD that incorporates momentum and adaptive learning rates for more efficient training:

Model Evaluation:

Model evaluation is the critical process of assessing how effectively the neural network generalizes to unseen data. It ensures that the model has learned the underlying geometric patterns rather than simply memorizing the training noise (overfitting).

1. **Classification Metrics (Categorical Results)**

   When your model predicts a category (e.g., "Cat" vs "Dog" or "Class 1" vs "Class 2"), we use these standard metrics:

   - **Accuracy: The percentage of total predictions that were correct.**

$$\textbf{Accuracy} = \frac{\textbf{\textit{Correct predictions}}}{\textbf{\textit{Total predictions}}}$$

- **Confusion Matrix:** A table that breaks down performance into four categories:
- **True Positives (TP):** Correctly predicted positive.
- **True Negatives (TN):** Correctly predicted negative.
- **False Positives (FP):** Predicted positive but was actually negative (Type I Error).
- **False Negatives (FN):** Predicted negative but was actually positive (Type II Error).
- **Precision:** Focuses on the quality of positive predictions (e.g., "When the model says it's a Cat, how often is it right?").
- **Recall (Sensitivity**): Focuses on the quantity of positives found (e.g., "Out of all the actual Cats, how many did the model find?").
- **F1-Score:** The balance between Precision and Recall. It is the best metric to use if your classes are imbalanced.

		Predicted condition	
	Total population = P + N	Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP)	False negative (FN)
	Negative (N)	False positive (FP)	True negative (TN)

Project Objectives and Tasks

The objective of this project is to classify data points into two classes, C1 and C2, which are distributed in a non-convex, interlocking pattern. I utilized a Multi-Layer Perceptron (MLP) to learn the decision boundary.

Key Tasks Performed:

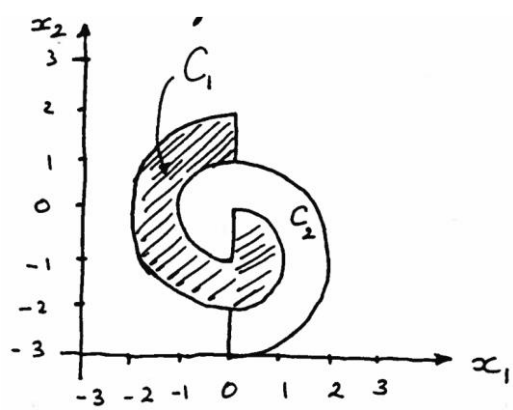
1. **Pattern Selection:** Generating and preparing a suitable pattern set for training.
2. **Evaluation**: Selecting a pattern set to evaluate the performance of the network after training.
3. **Visualization**: Ploting the performance (learning curves) for the training phase.

4. **Parametric Study:** Conducting a detailed study on how the following parameters affect performance and convergence speed:
    - Number of neurons in the hidden layer.
    - Number of hidden layers.
    - Learning rate and momentum coefficients.
    - The effect of data normalization.
    - Different Data set volumes.

Methodology

Pattern Selection:

To start the project, I first had to create a dataset that matches the non-convex decision regions shown in the problem description. Since the classes C1 and C2 are interlocked in a complex way, I used a **rejection sampling** method to generate the points.



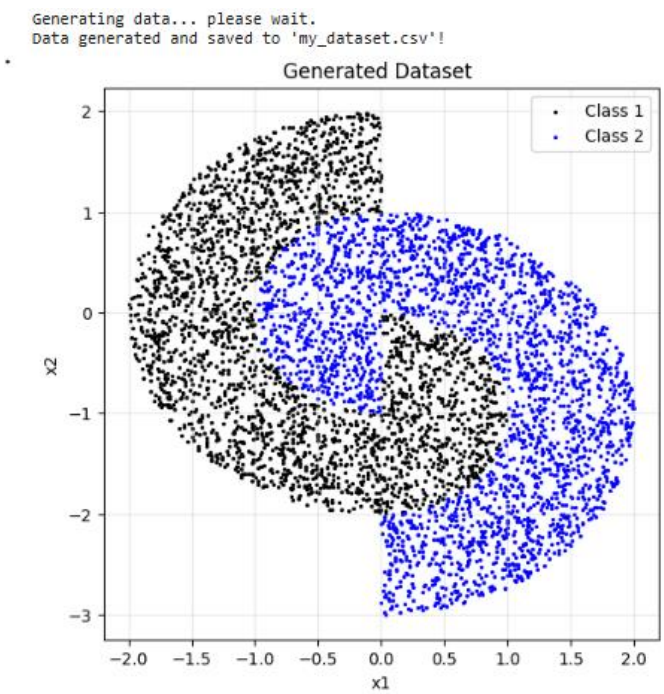
I wrote a script to pick random (x1, x2) points between -3 and 3. For each point, I checked its distance from two different centers to see if it fell into the right regions based on the following logic:

Class C1 :

$$\boldsymbol{C1 : \{(1 \le x_1^2 + x_2^2 \le 4) and\ x_1 \le 0\} \cup \{(x_1^2 + (x_2+1)^2 \le 1) and\ x_1 > 0\}}$$

Class C2 :

$$C2: \{(x_1^2 + x_2^2 < 1) and\ x_1 \leq 0\} \cup \{(1 \leq x_1^2 + (x_2 + 1)^2 \leq 4) and\ x_1 > 0\}$$



I repeated this sampling process until I collected a total of 7,000 valid samples (approximately 3,500 points per class). This ensures that the dataset is balanced and provides enough information for the neural network to learn the non-linear boundaries. The generated data was then visualized to verify it matches the problem set and saved into a CSV file for the training phase.

Data Preprocessing and Normalization

After generating the raw dataset, since the data generation process utilized random sampling within a loop, the resulting dataset was naturally shuffled, ensuring a random distribution of classes.

Data Splitting Strategy: To ensure a rigorous evaluation and prevent **data leakage**, the splitting was performed before any normalization steps. The data was partitioned into three subsets:

- **Training Set (70%):** Used for weight updates and model learning.
- **Validation Set (10%):** Used for hyperparameter tuning and selecting the best model configuration.
- **Test Set (20%):** Held out until the very end to provide an unbiased evaluation of the final model's performance.

Standardization Methodology:

Standardization was handled carefully to maintain the "unseen" nature of the validation and test data. A StandardScaler was initialized and fitted only on the Training Set to calculate the mean and standard deviation. These training-derived parameters were then used to transform the validation and test sets. This guarantees that the model remains completely unaware of the statistical distribution of the test data during the training phase.

2.2. Network structure and the hyperparameters

To determine the optimal architecture for classifying the non-convex dataset, I conducted a systematic parametric study. Initially, I made a baseline Multi-Layer Perceptron (MLP) and observed the effects of varying key hyperparameters on the learning curves (Training and Validation Loss).

The following parameters were investigated:

- **Number of Neurons in the Hidden Layer:** We experimented with configurations of **[4, 8, 10, 15, 20]** neurons to find the balance between model capacity and generalization.
- **Number of Hidden Layers:** The network depth was tested using **[1, 2, 3, 4,5]** hidden layers.
- **Learning Rate (η):** We tested values of **[0.001, 0.01, 0.1, 1.0, 10.0]** to observe convergence speed and stability.
- **Momentum Coefficient (α):** We evaluated the impact of momentum using values of **[0.0, 0.5, 0.9]** to help the optimizer navigate the loss surface.

- **Activation Functions:** We compared **ReLU**, **Tanh**, and **Sigmoid** functions to determine which provides the best gradient flow for this specific interlocking pattern.
- **Different dataset volumes:** We compared 3 datasets with the size of 3500,7000,15000.

Hyperparameter Tuning Strategy

Due to the vast search space of hyperparameters, I have selected this approach to identify the best model configuration:

- **Phase 1: Hyperparameter Sensitivity and Depth Exploration:** For 1 hidden layer configuration (L=1), a parametric study was conducted to find the most effective learning rate, momentum, and activation function. By fixing the depth of the network, I was able to isolate the sensitivity of the model to individual parameters. After that, a comparative study was performed by varying the number of hidden layers from 1 to 5. This phase evaluated whether increasing the model's capacity significantly improved the separation of the interlocking non-convex boundaries.
- **Phase 2: Layer Capacity Fine-tuning:** After finding the best parameter I made a comparison model with the best hidden layer number, activation function, momentum and learning rate to find best number of neurons in each layer.
- **Phase 3: Importance of Data Pre-processing:** Comparative study on Normalization.
- **Phase 4: Final Model Integration & Testing:** The absolute best combination of hyperparameters was used to train the final model. This model was then evaluated on the **completely unseen normal Test Set and also a critical points test set** to report the final accuracy and generate the confusion matrix.
- **Phase 5: Scalability Study:** Evaluating the model's performance on larger or smaller datasets (3.5k, 15k).

Results & Discussion (Parametric Study)

Phase 1: Hyperparameter Sensitivity and Depth Exploration

In the first phase of the study, a systematic evaluation was performed to identify how individual hyperparameters influence the network's learning behavior. To achieve this, a Control Variable strategy was used: a baseline model was made, and in each experiment,

some parameters were kept constant while one specific parameter was varied across a predefined range.

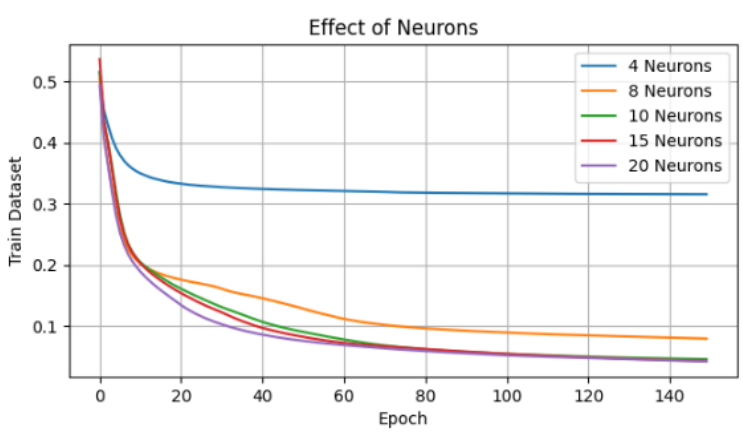
Baseline Configuration:

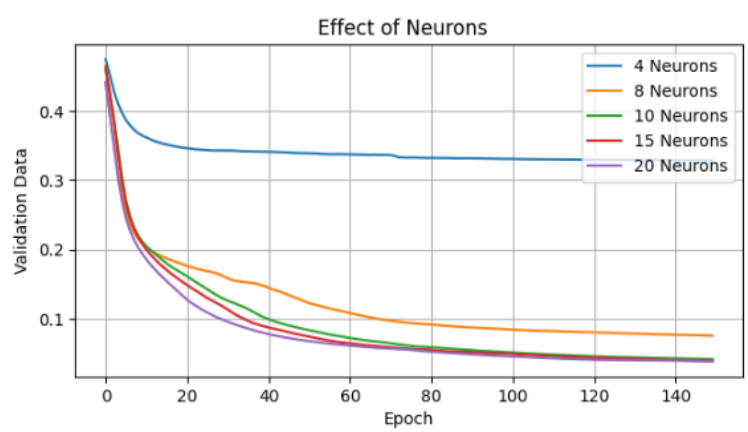
- Architecture: the base model configuration was set to layers=1 (a single hidden layer)
- Neurons: 15 per hidden layer.
- Learning Rate (η): 0.01.
- Momentum (α): 0.9.
- Activation: ReLU.
- Dataset Volume**:** 7,000 samples.
- Epochs = 150
- Batch size = 32

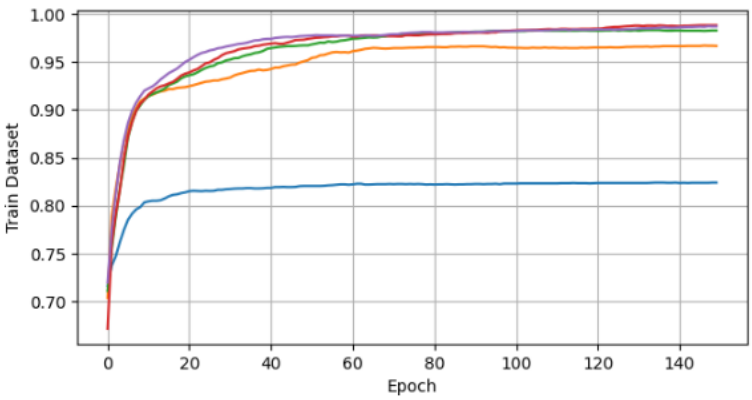
Experiment 1: Hidden Layer Capacity (Neurons)

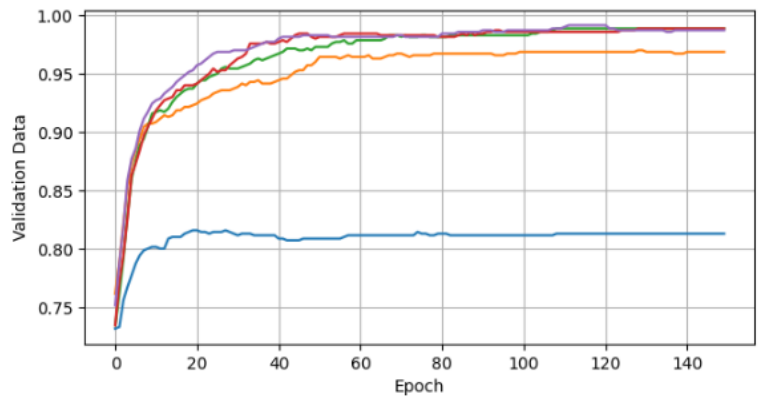
To study the effect of model capacity on learning speed, the number of neurons in the hidden layers was varied: n ∈ $\{4, 8, 10, 15, 20\}$.

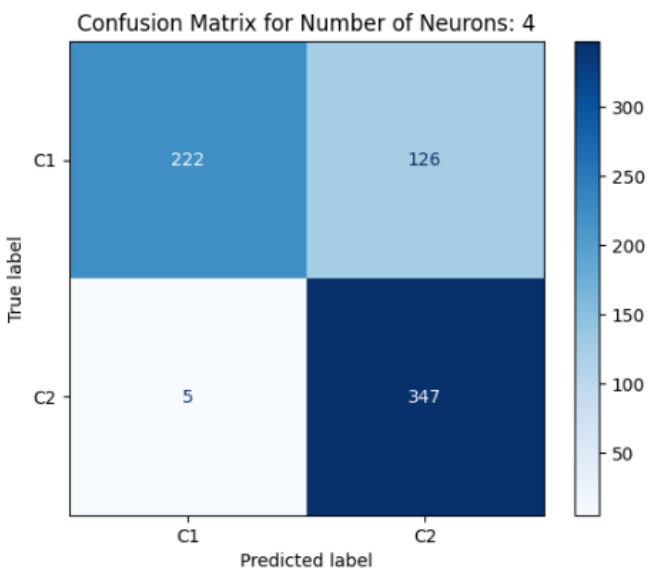
- **Fixed Parameters**: **hidden layers** = 1 $\boldsymbol{\eta}$ = 0.01 , $\boldsymbol{\alpha}$ = 0.9, **activation** = ReLU, **epochs** = 150, **batch_size** = 32 , **Dataset Volume:** 7,000 samples , **Normalization**.
- **Objective:** To determine the minimum number of neurons required to capture the interlocking spiral pattern without underfitting.



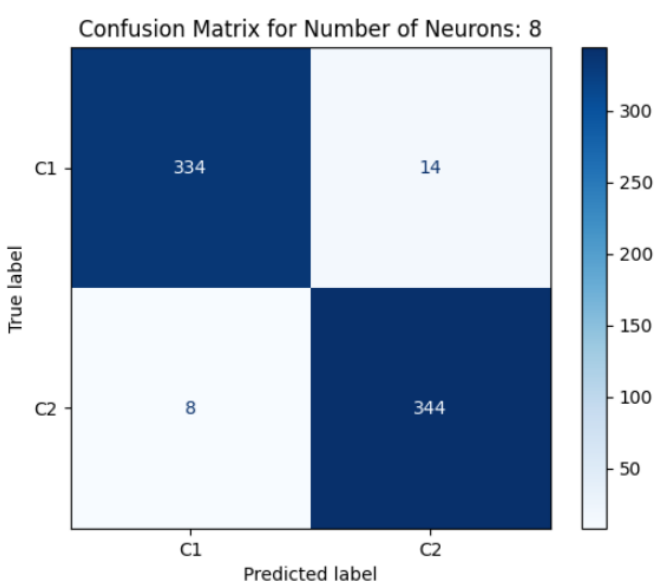




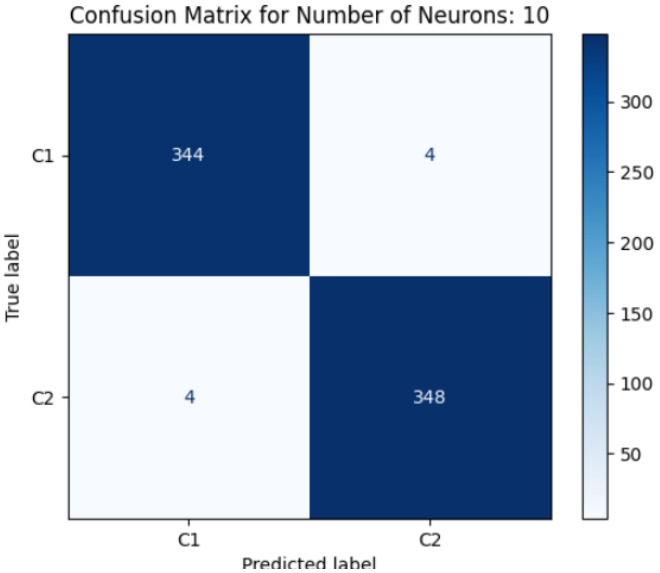




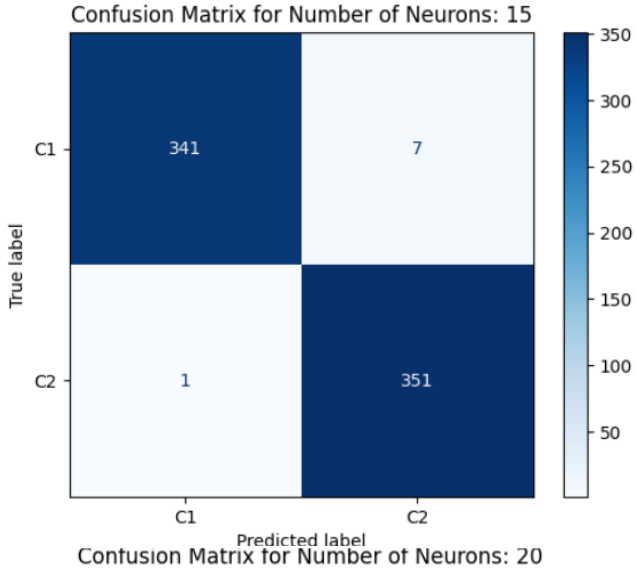

```
classification_report for 4 Neurons:
              precision    recall  f1-score   support

           0       0.98      0.64      0.77       348
           1       0.73      0.99      0.84       352

    accuracy                           0.81       700
   macro avg       0.86      0.81      0.81       700
weighted avg       0.86      0.81      0.81       700
```



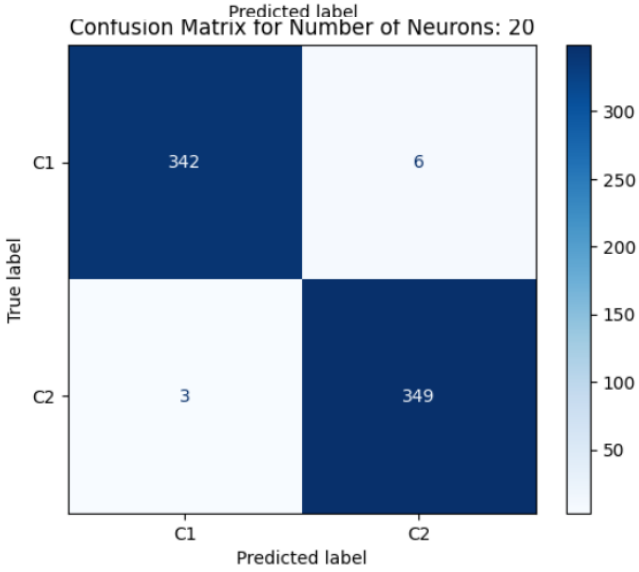

```
classification_report for 8 Neurons:
              precision    recall  f1-score   support

           0       0.98      0.96      0.97       348
           1       0.96      0.98      0.97       352

    accuracy                           0.97       700
   macro avg       0.97      0.97      0.97       700
weighted avg       0.97      0.97      0.97       700
```



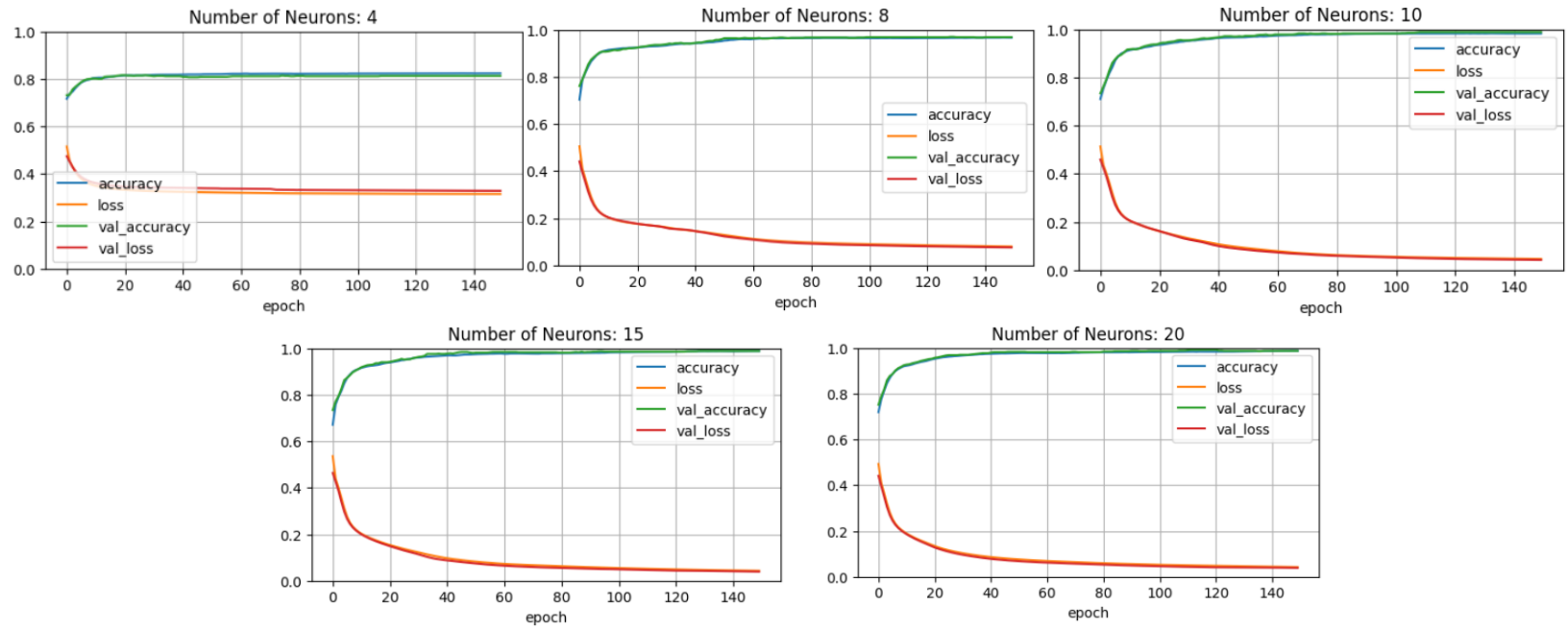

```
classification_report for 10 Neurons:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       348
           1       0.99      0.99      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```




```
classification_report for 15 Neurons:
              precision    recall  f1-score   support

           0       1.00      0.98      0.99       348
           1       0.98      1.00      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```




```
classification_report for 20 Neurons:
              precision    recall  f1-score   support

           0       0.99      0.98      0.99       348
           1       0.98      0.99      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```



The results demonstrate a clear transition from underfitting to performance saturation.

We started with a simple **4-neuron** model that was too basic to understand the complex geometry (Underfitting), hitting only **81% accuracy**. The model is honestly just guessing, it can see the big picture but fails miserably at the "tricky" parts where the circles overlap. Specifically, it keeps confusing Class 0 for Class 1.

By increasing the complexity to 15 neurons, a critical jump in performance occurs at **8 neurons**, where **accuracy** surges to **97%**. The model successfully begins to approximate the circular arcs of both classes

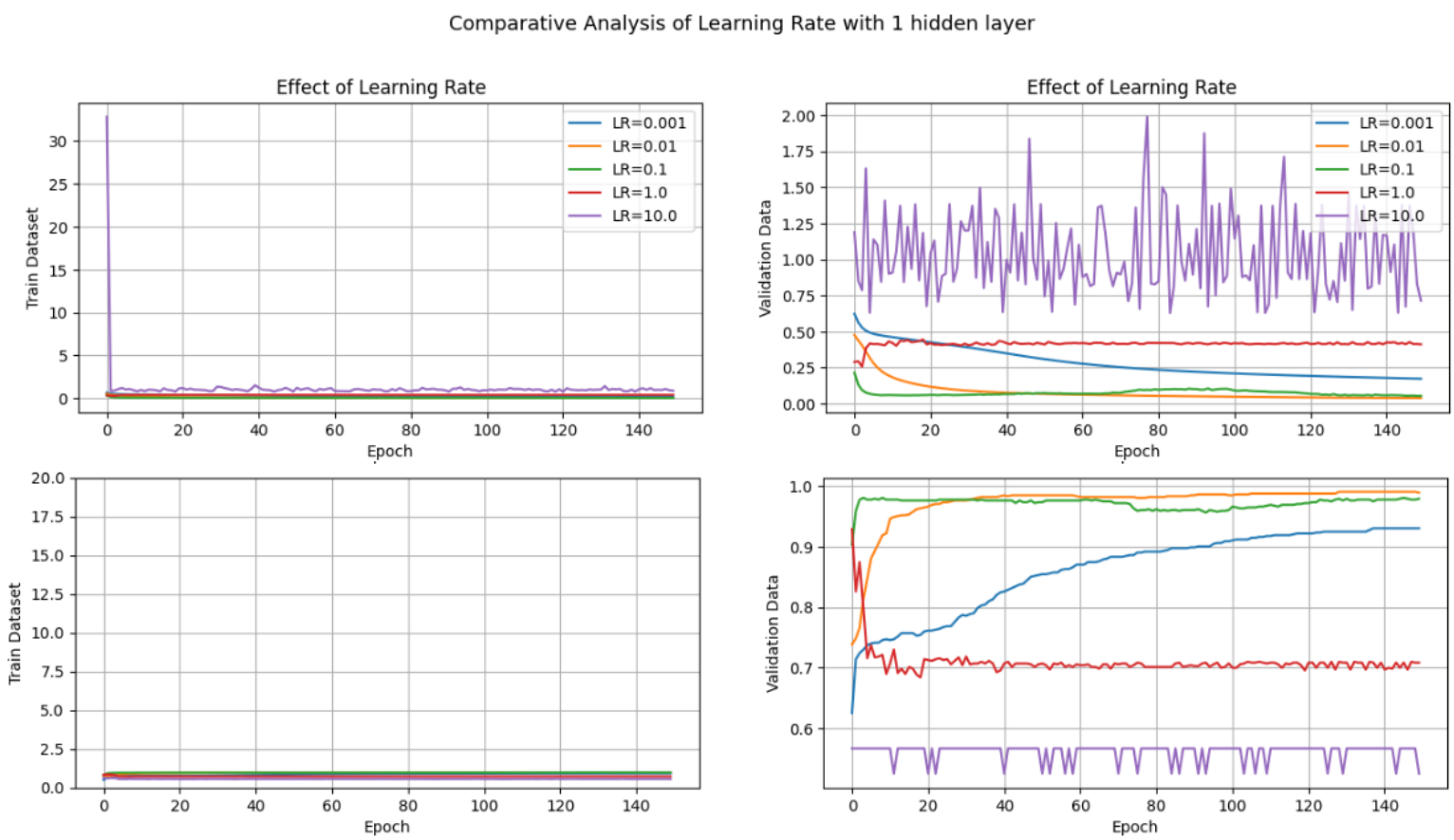
Beyond **10 neurons**, **accuracy** stabilizes at **99%** (the model enters **saturation** at 99% accuracy.). While increasing neurons to 15 or 20 provides smoother convergence in the learning curves, suggesting that a 10-15 neuron configuration is the "sweet spot" for efficiency. Adding more neurons beyond this (like going to 20) doesn't really give you much more information.

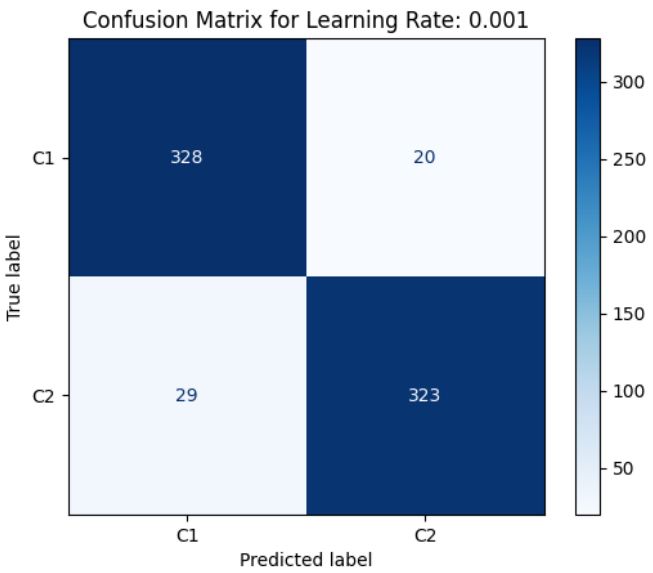
I would select **15 neurons** for the final model, as it achieves the **highest accuracy (99%)** while showing a **more stable and smoother learning behavior** compared to the 10-neuron configuration. In addition, the extra capacity helps the model remain robust when scaling to larger datasets in **Phase 5**.

Experiment 2: Learning Rate Sensitivity

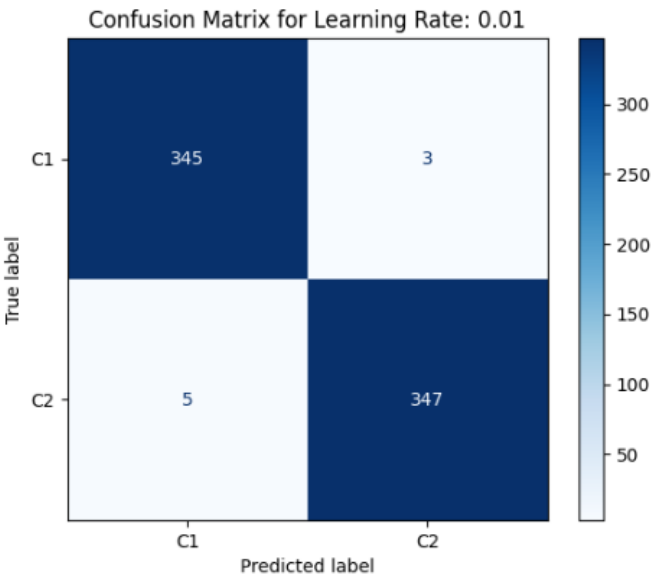
The learning rate was tested across several orders of magnitude: $\eta \in \{0.001, 0.01, 0.1, 1.0, 10.0\}$.

- **Fixed Parameters: neurons** =15, **hidden layers** = 1, α = 0.9 , **activation** = ReLU, **epochs** = 150, **batch_size**=32 , **Dataset Volume:** 7,000 samples , **Normalization**.
- **Objective:** To identify the "sweet spot" where the model converges rapidly without overshooting the global minimum or becoming unstable.

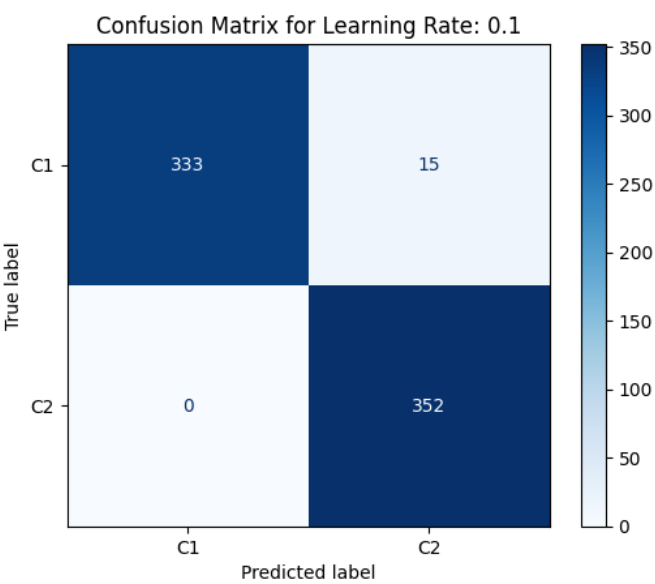




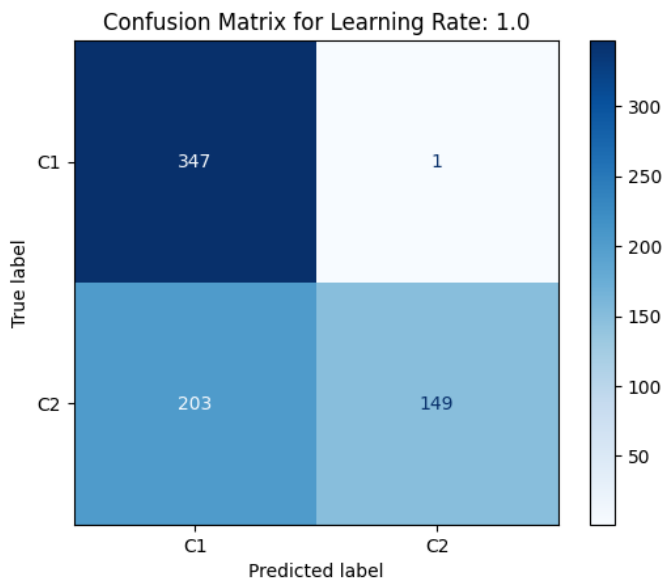

```
classification_report for Learning Rate 0.001
:              precision    recall  f1-score   support

           0       0.92      0.94      0.93       348
           1       0.94      0.92      0.93       352

    accuracy                           0.93       700
   macro avg       0.93      0.93      0.93       700
weighted avg       0.93      0.93      0.93       700
```



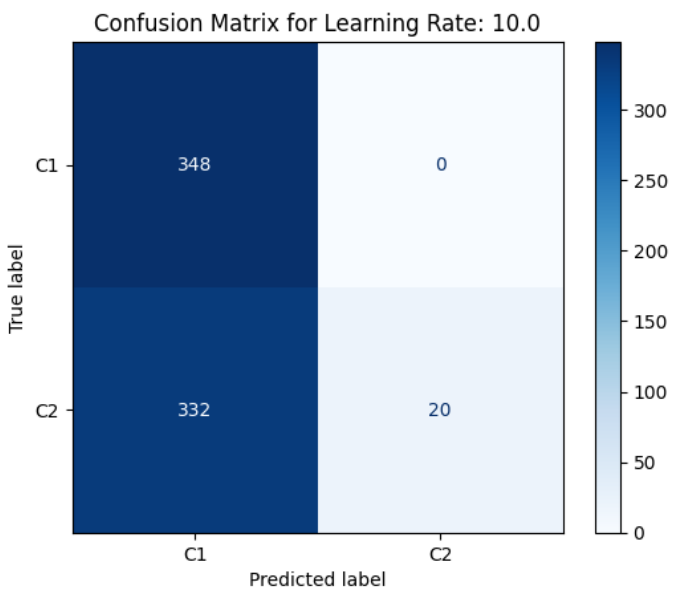

```
classification_report for Learning Rate 0.01
:              precision    recall  f1-score   support

           0       0.99      0.99      0.99       348
           1       0.99      0.99      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```



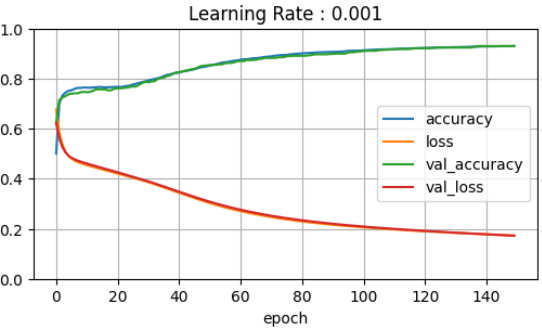

```
classification_report for Learning Rate 0.1
:              precision    recall  f1-score   support

           0       1.00      0.96      0.98       348
           1       0.96      1.00      0.98       352

    accuracy                           0.98       700
   macro avg       0.98      0.98      0.98       700
weighted avg       0.98      0.98      0.98       700
```

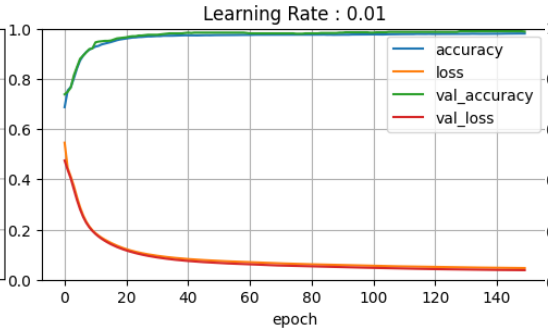


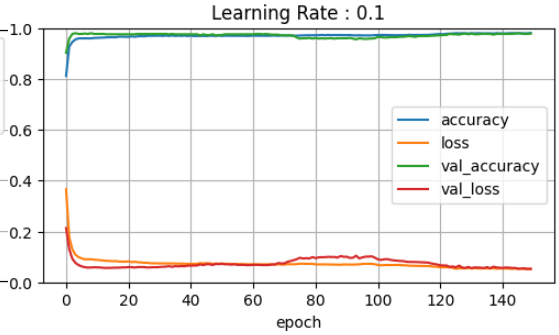

```
classification_report for Learning Rate 1.0
:              precision    recall  f1-score   support

           0       0.63      1.00      0.77       348
           1       0.99      0.42      0.59       352

    accuracy                           0.71       700
   macro avg       0.81      0.71      0.68       700
weighted avg       0.81      0.71      0.68       700
```

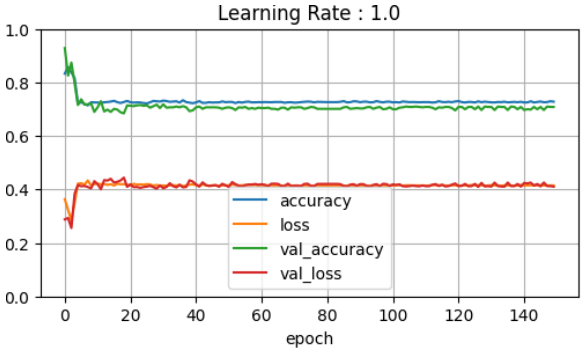


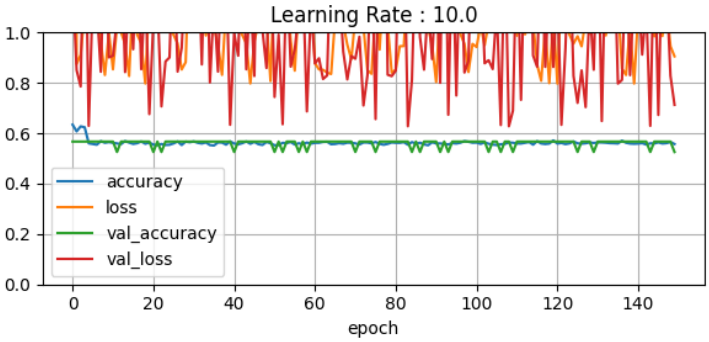

```
classification_report for Learning Rate 10.0
:              precision    recall  f1-score   support

           0       0.51      1.00      0.68       348
           1       1.00      0.06      0.11       352

    accuracy                           0.53       700
   macro avg       0.76      0.53      0.39       700
weighted avg       0.76      0.53      0.39       700
```











The curves for **LR=0.001** (Blue line) are smooth. The model is "being too careful." It's learning, but it's taking tiny baby steps. So, after 150 epochs, it only reached **93% accuracy**. It hasn't finished the job; given more epochs, it might get there, but it's inefficient. We can see **49 errors** in the confusion matrix.

LR = 0.01 is perfectly balanced. It's fast enough to reach the global minimum quickly but stable enough to stay there without bouncing around. It hit a near-perfect **99% accuracy** with **only 8 total errors**. This is the winner for the final model .

LR = 0.1 is extremely fast convergence, hitting peak accuracy in almost 10 epochs. While it achieves **98% accuracy**, we can see the validation loss starts to "jump" slightly at the end because its steps are a bit too large. It made **15 errors**, nearly double the errors of the 0.01 rate.

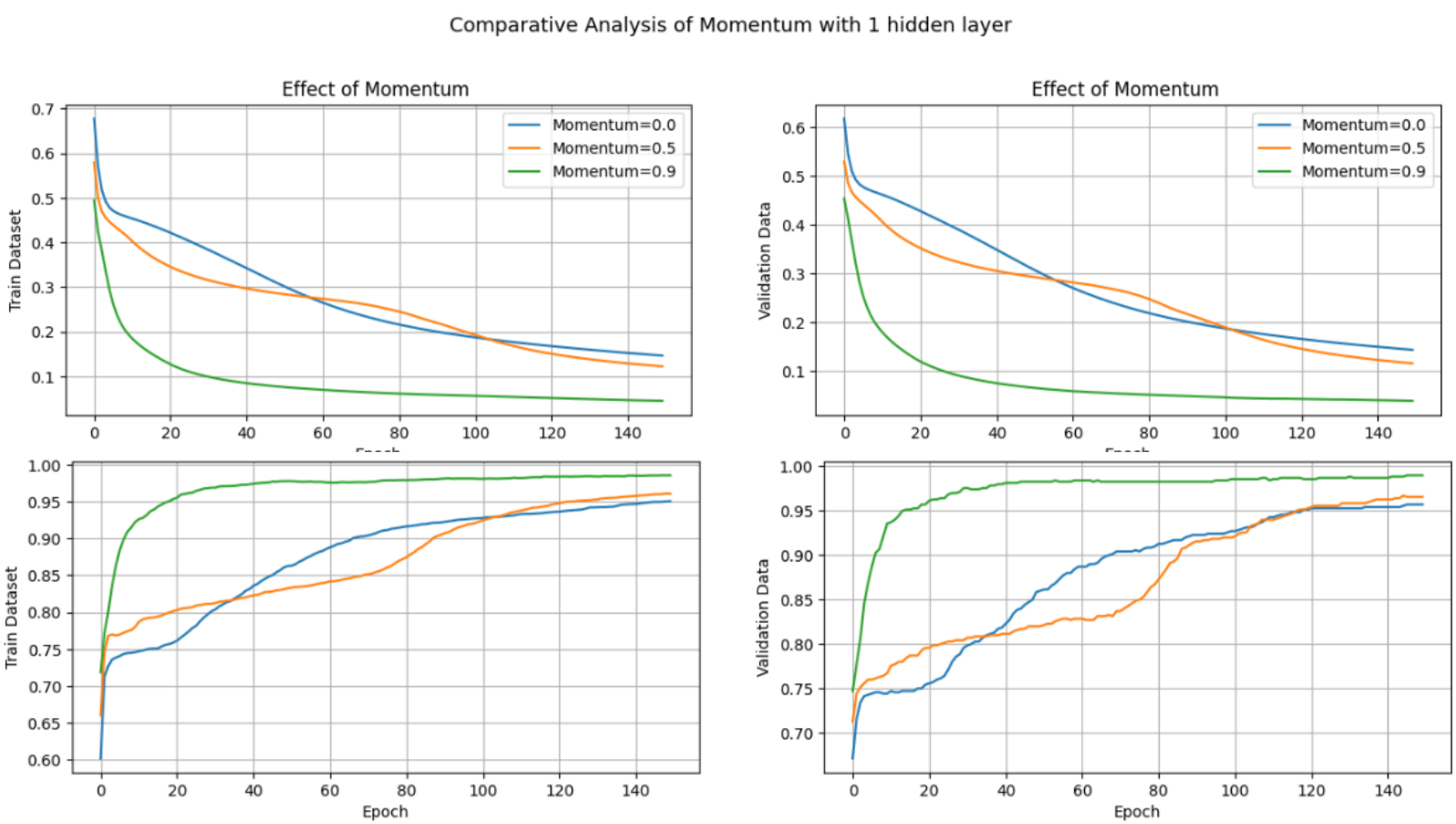
The model with **LR = 1.0** is stuck. Accuracy flatlines at **71%**. The steps are so large the model is just jumping back and forth across the valley of the loss function, unable to settle in the middle.

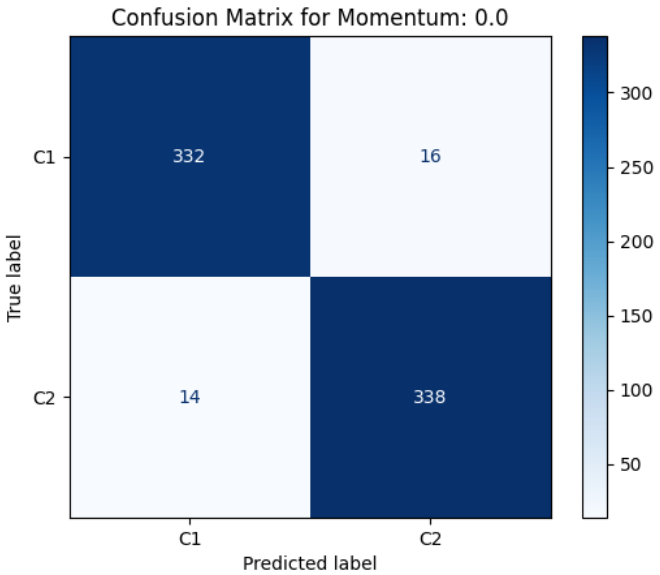
At **LR = 10.0**, The loss looks like a heart monitor fluctuating wildly. The steps are so violent the model is literally flying out of the valley entirely. The confusion matrix shows **53% accuracy**, which is basically random guessing for a two-class problem.

Experiment 3: Momentum Impact

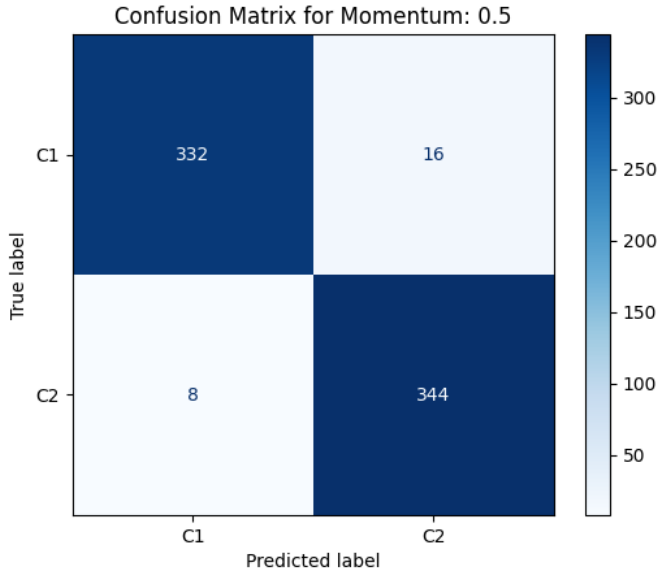
The momentum coefficient was varied to observe its effect on smoothing the optimization path: $\boldsymbol{\alpha} \in$ **{0.0, 0.5, 0.9}**

- **Fixed Parameters: neurons** = 15, **hidden layers** = 1, $\boldsymbol{\eta}$ = 0.01 , **activation** = ReLU, **epochs** = 150, **batch_size** = 32 , **Dataset Volume:** 7,000 samples, **Normalization**.
- **Objective:** The goal of this experiment is to see how adding momentum helps the SGD optimizer move more smoothly during training and improves convergence when the loss surface is complex and non-linear.

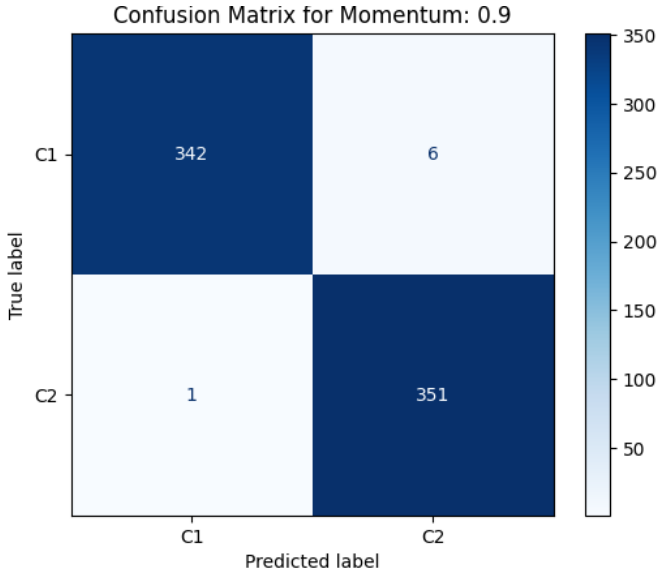




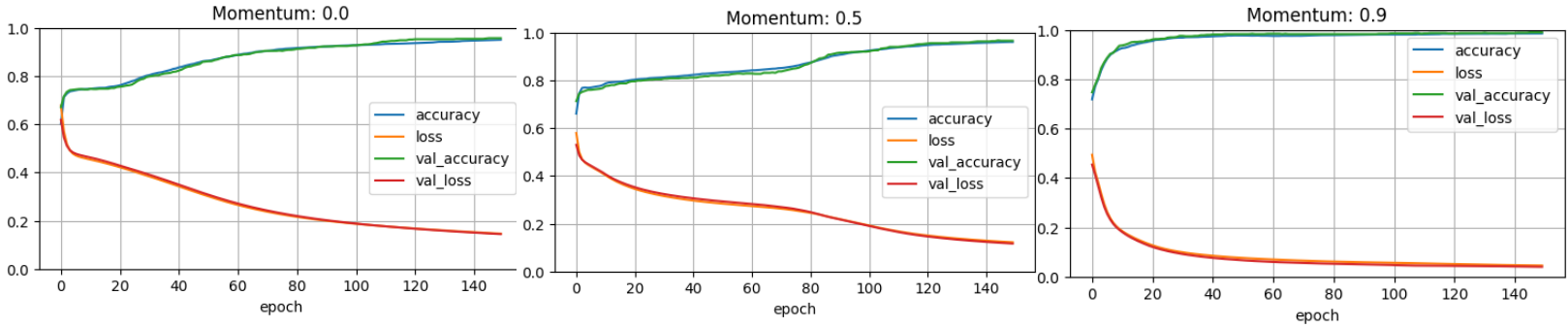

```
classification_report for 0.0 :
              precision    recall  f1-score   support

           0       0.96      0.95      0.96       348
           1       0.95      0.96      0.96       352

    accuracy                           0.96       700
   macro avg       0.96      0.96      0.96       700
weighted avg       0.96      0.96      0.96       700
```




```
classification_report for 0.5 :
              precision    recall  f1-score   support

           0       0.98      0.95      0.97       348
           1       0.96      0.98      0.97       352

    accuracy                           0.97       700
   macro avg       0.97      0.97      0.97       700
weighted avg       0.97      0.97      0.97       700
```




```
classification_report for 0.9 :
              precision    recall  f1-score   support

           0       1.00      0.98      0.99       348
           1       0.98      1.00      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```



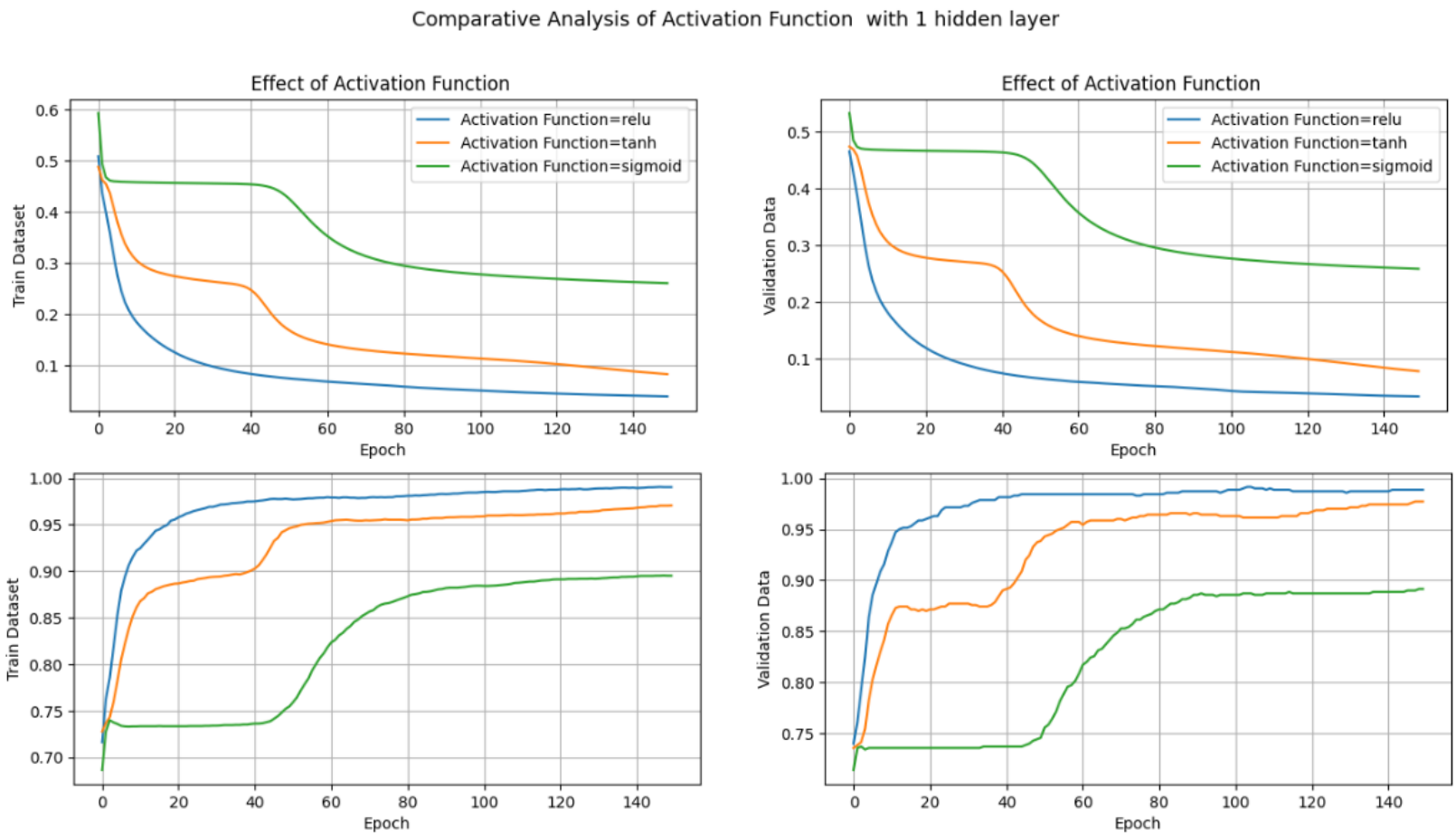
Without momentum, the optimizer only looks at the current gradient. it's safe, but very slow. It made **30 total errors** (16 in Class C1 and 14 in Class C2). It takes almost the entire 150 epochs to settle to finish with **96% accuracy** .

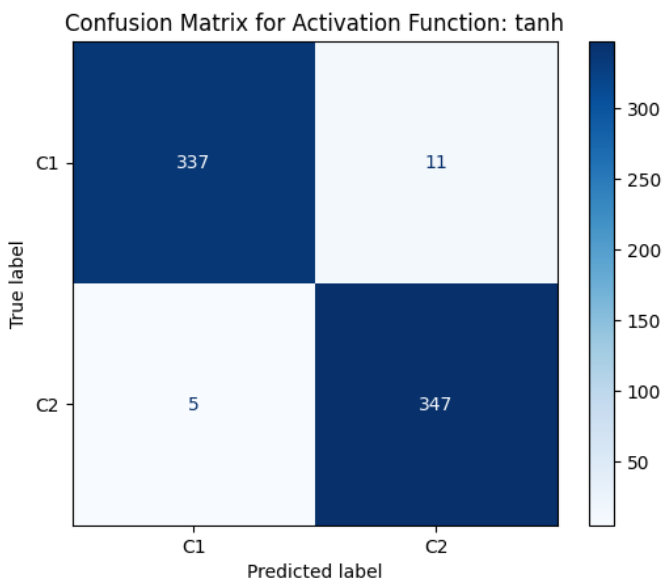
When we have 15 neurons and a complex interlocking pattern, the "surface" of the loss function is very bumpy. High momentum(**momentum = 0.9**) ensures that the SGD optimizer doesn't get distracted by small ripples and stays focused on reaching the absolute lowest point of error. The fact that it achieved **99% accuracy** with almost 6 errors in Class C2 (only 1 mistake!) is proof that it is perfect.

Experiment 4: Activation Function Comparison

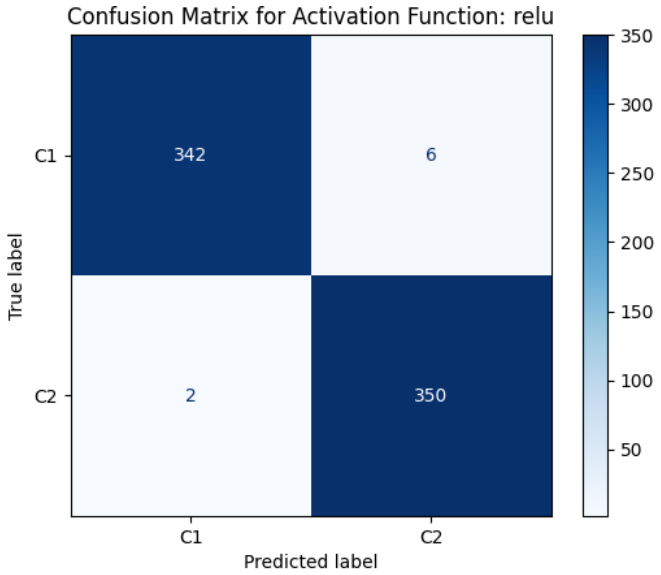
The model was trained using three different non-linear activation functions: ReLU, Tanh, and Sigmoid.

- **Fixed Parameters: neurons** = 15, **hidden layers** = 1, $\boldsymbol{\eta}$ = 0.01, $\boldsymbol{\alpha}$ = 0.9, **epochs** = 150, **batch_size** = 32 , **Dataset Volume:** 7,000 samples, **Normalization** .
- **Objective:** The goal of this experiment is to compare how different activation functions .

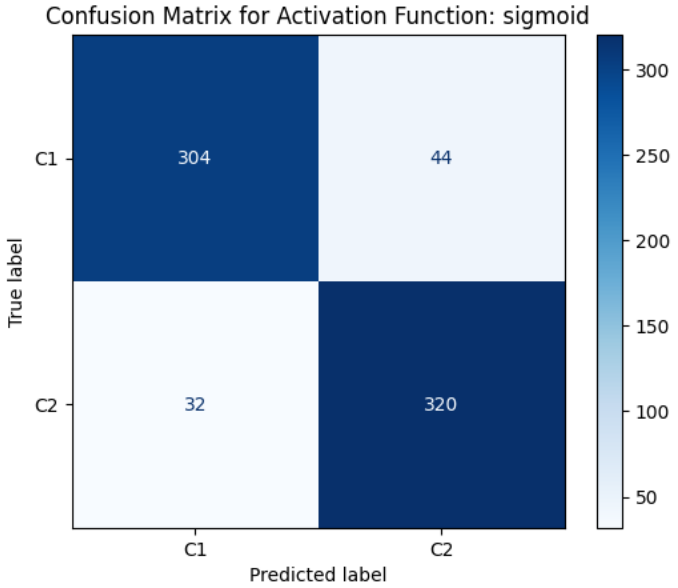




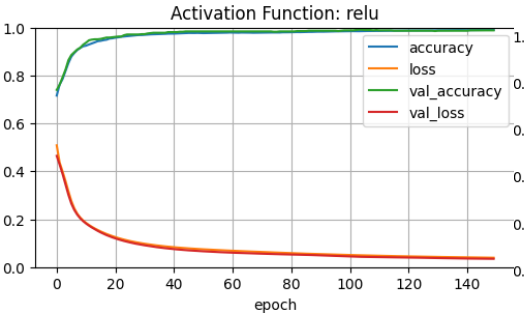

```
classification_report for tanh :
              precision    recall  f1-score   support

           0       0.99      0.97      0.98       348
           1       0.97      0.99      0.98       352

    accuracy                           0.98       700
   macro avg       0.98      0.98      0.98       700
weighted avg       0.98      0.98      0.98       700
```

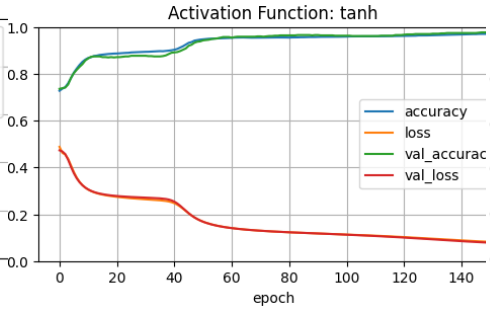


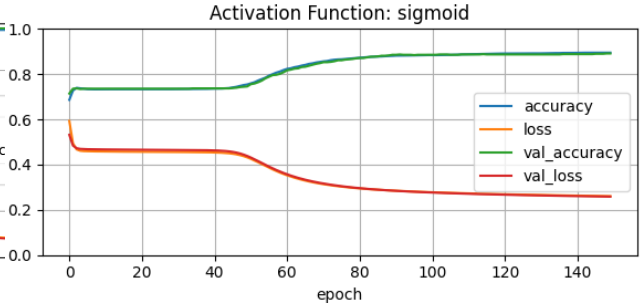

```
classification_report for relu :
              precision    recall  f1-score   support

           0       0.99      0.98      0.99       348
           1       0.98      0.99      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```




```
classification_report for sigmoid :
              precision    recall  f1-score   support

           0       0.90      0.87      0.89       348
           1       0.88      0.91      0.89       352

    accuracy                           0.89       700
   macro avg       0.89      0.89      0.89       700
weighted avg       0.89      0.89      0.89       700
```







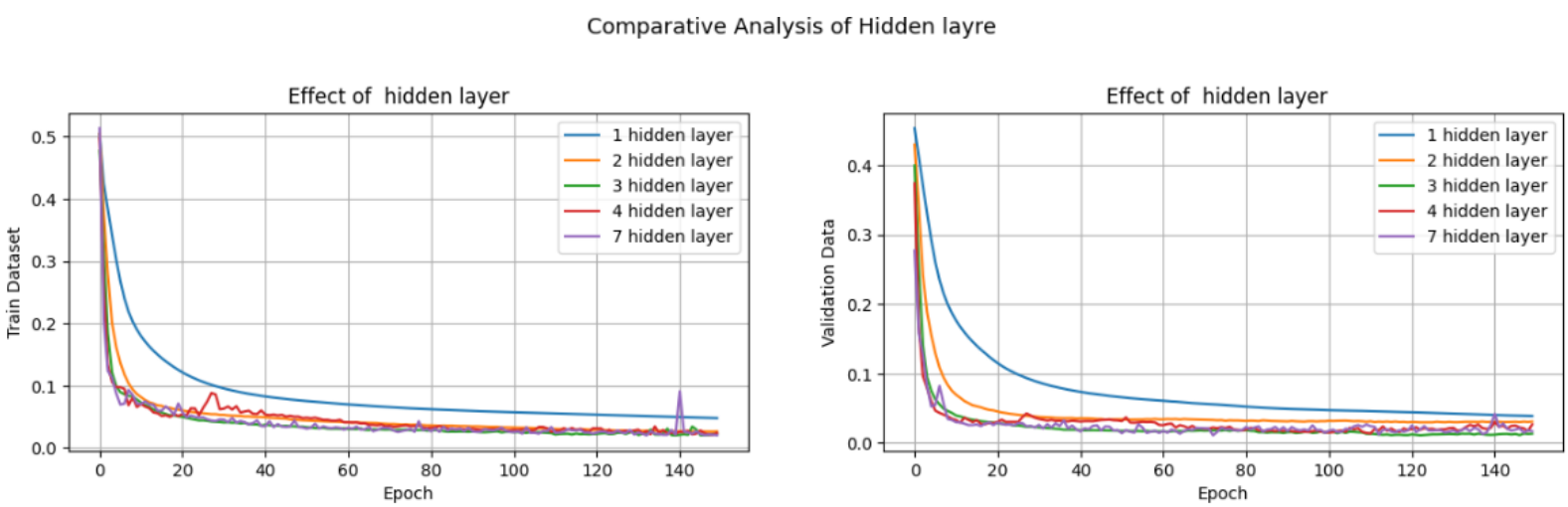
ReLU is the clear winner for this 15-neuron architecture. While **Sigmoid** struggled with vanishing gradients and couldn't resolve the interlocking circles effectively (because the Sigmoid function squashes its inputs so much, the signals for the model to learn become tiny and almost vanish, making the training painfully slow with **89% accuracy**), **ReLU** achieved near-perfect results **(99%)** almost immediately.

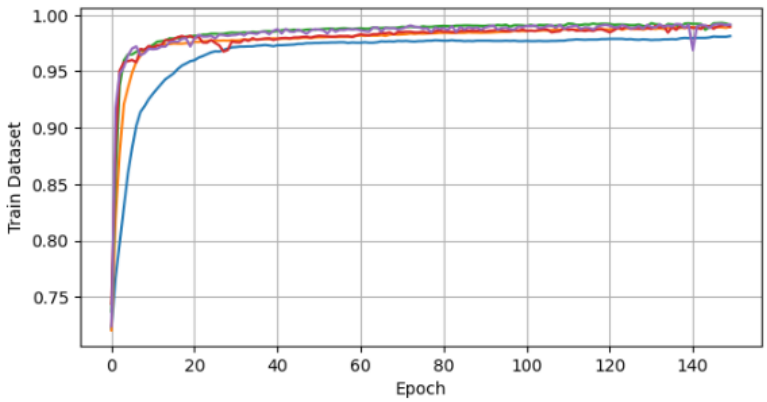
On the other hand, **tanh** is better than Sigmoid, but it's still mathematically heavier than ReLU. It eventually gets there, but it takes twice as long. It reached **98% accuracy with 16 total errors**. It's less efficient than ReLU.

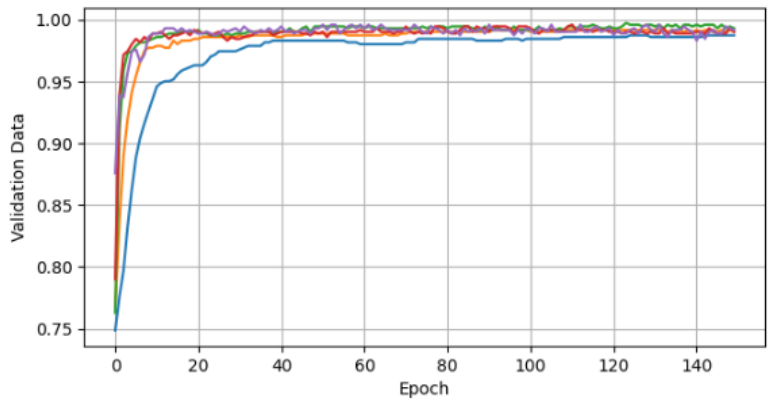
Experiment 5: Number of Hidden Layers Comparison

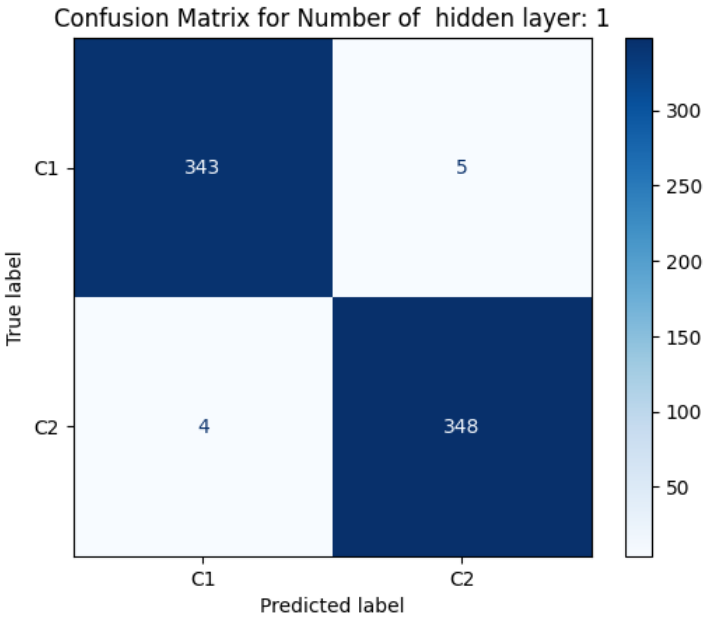
The model was trained using three different hidden layers ∈ **{1,2,3, 4, 7}**

- **Fixed Parameters**: **neurons** = 15, $\boldsymbol{\eta}$ = 0.01, $\boldsymbol{\alpha}$ = 0.9 , **activation** = ReLU , **epochs** = 150, batch_size=32 , **Dataset Volume:** 7,000 samples , **Normalization**.
- **Objective:** The goal of this experiment is to analyze how increasing the number of hidden layers affects the model's performance.

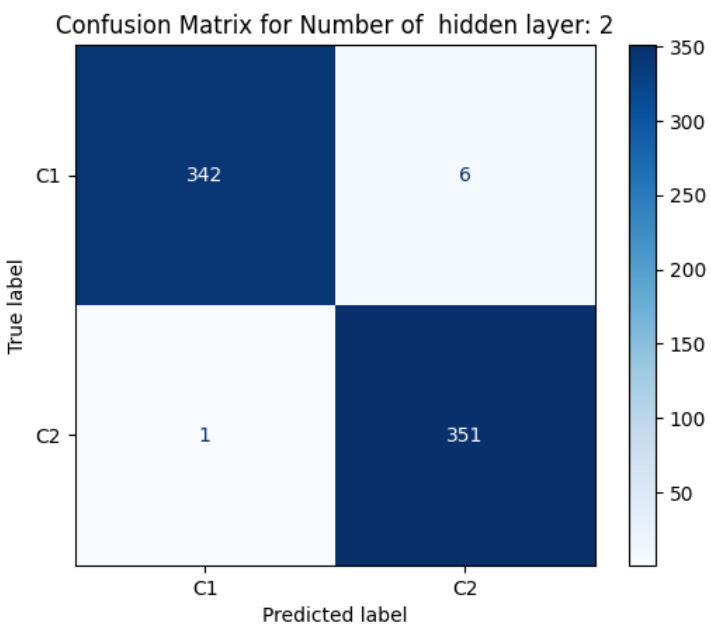




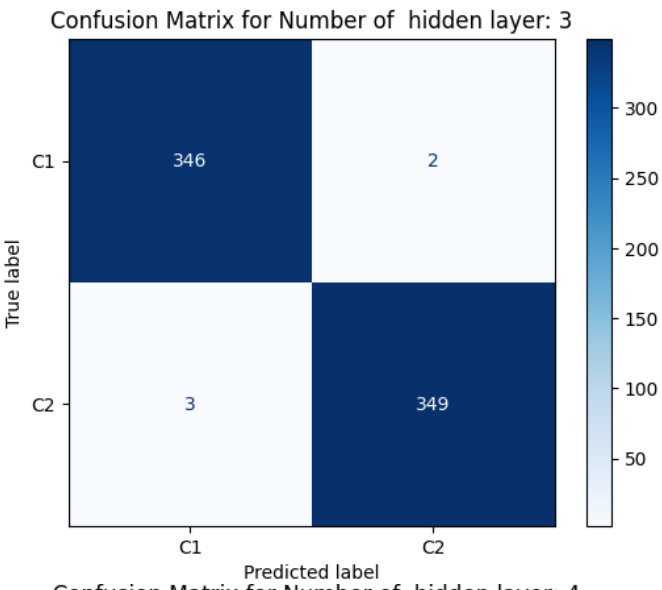




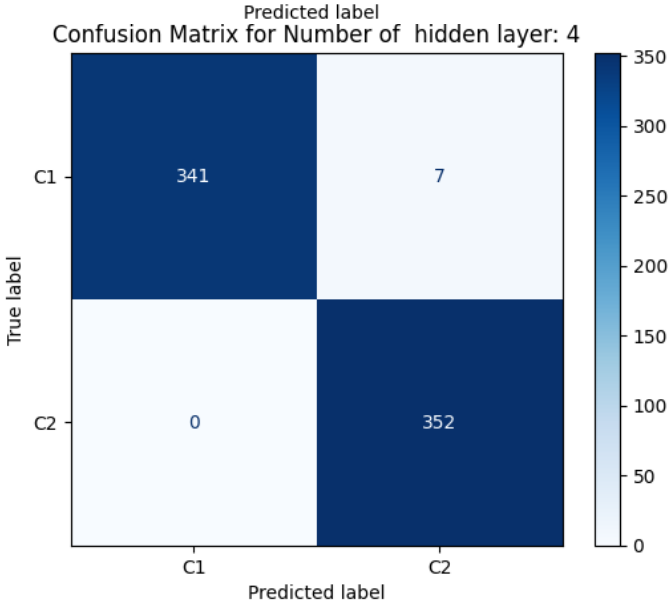

```
classification_report for 1 hidden layer:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       348
           1       0.99      0.99      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```



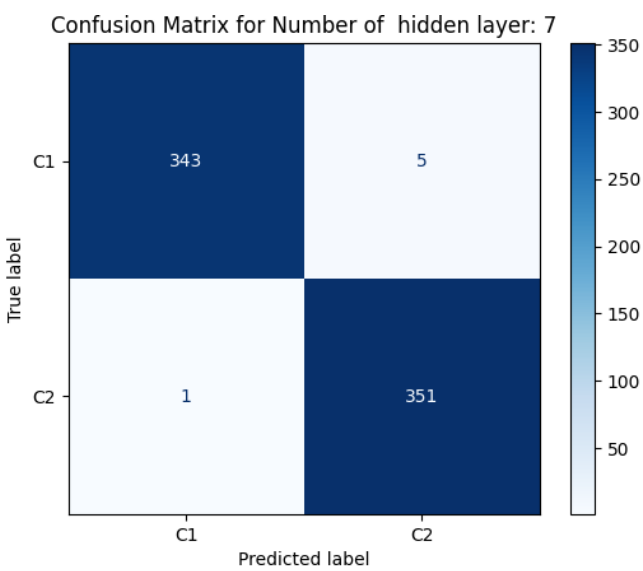

```
classification_report for 2 hidden layer:
              precision    recall  f1-score   support

           0       1.00      0.98      0.99       348
           1       0.98      1.00      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```



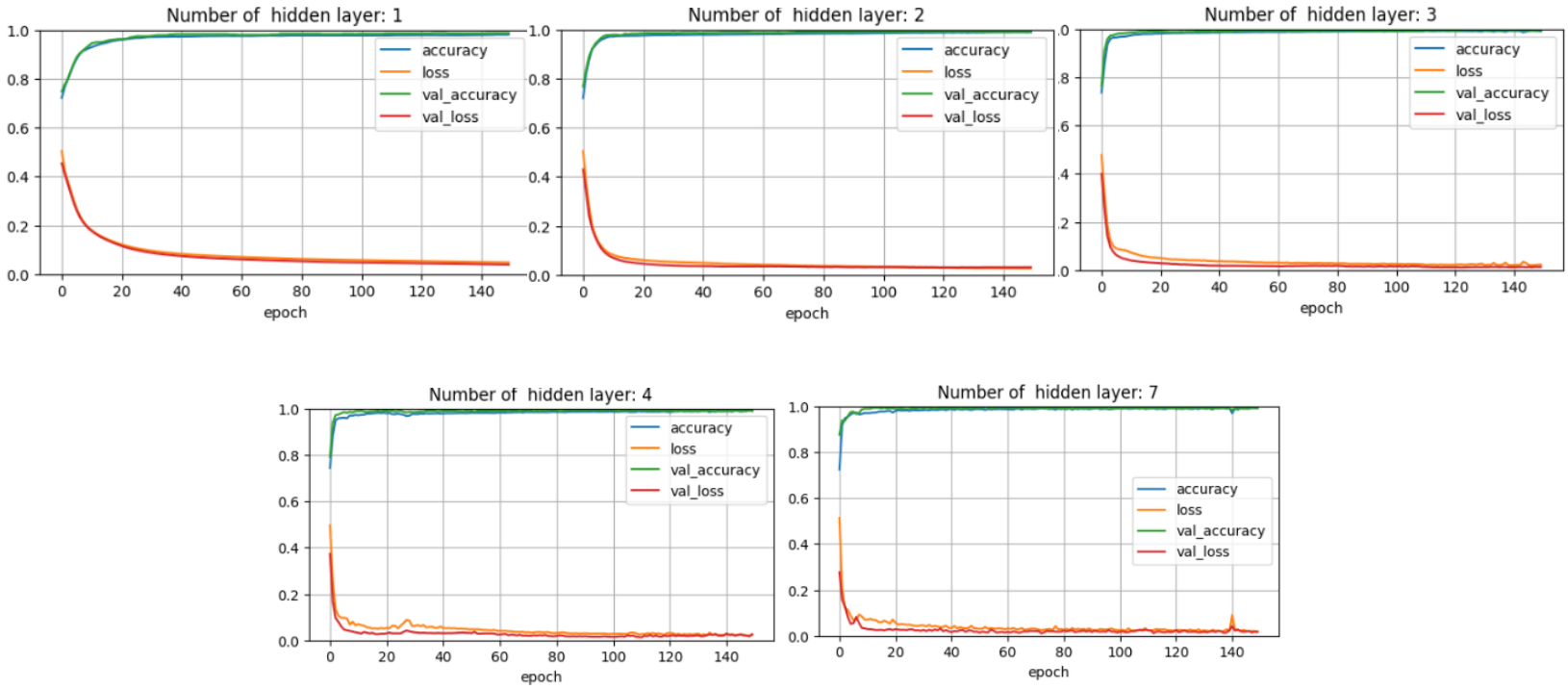

```
classification_report for 3 hidden layer:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       348
           1       0.99      0.99      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```




```
classification_report for 4 hidden layer:
              precision    recall  f1-score   support

           0       1.00      0.98      0.99       348
           1       0.98      1.00      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```




```
classification_report for 7 hidden layer:
              precision    recall  f1-score   support

           0       1.00      0.99      0.99       348
           1       0.99      1.00      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```



1 Hidden Layer starts much lower (**~75% accuracy**) and takes a long, slow path to reach the top. While it eventually gets to 99%, it's the most inefficient during the training phase (It's too simple).

Multi-Layer (**2, 3, 4, 7 Layers**): As soon as we add that second layer, the model wakes up **(99%).** These models all drop their loss almost vertically in the first 10 epochs. Having more than one layer allows the network to learn more , which is crucial for those interlocking circles.

If we look at the confusion matrices, we can actually pick a definitive winner:

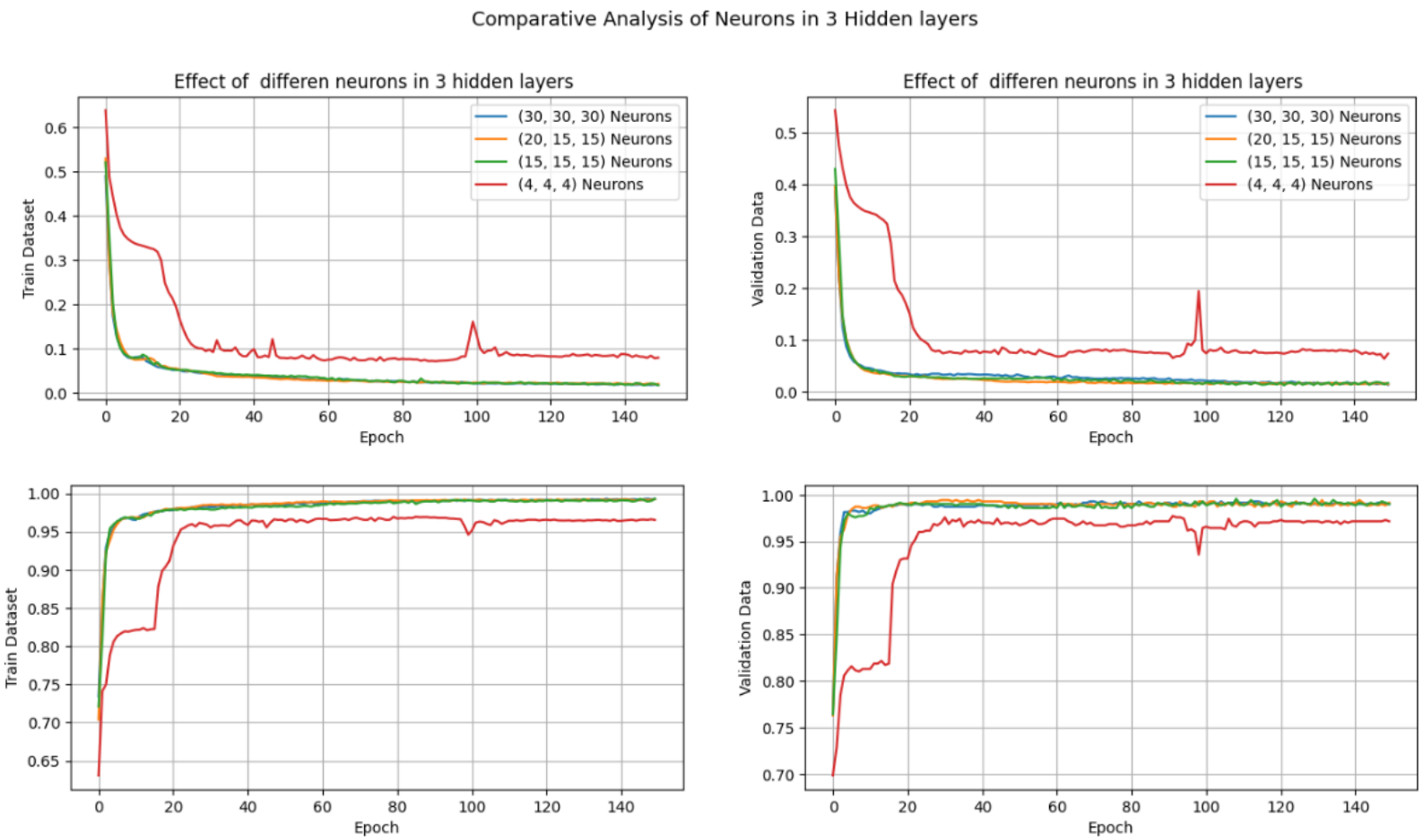
The **3-layer** model is our absolute result. It achieved the lowest **total error count(5) with accuracy (99%).** and shows the most balanced performance between the two classes.

With **7 layers**, if we look closely at the training loss, there is a sharp, weird spike right near epoch 140. This indicates when a model is too deep for a relatively simple 2D problem, it can start to **overfit** or the gradients can become noisy. It actually made more **mistakes (6)** than the 3-layer model (5).

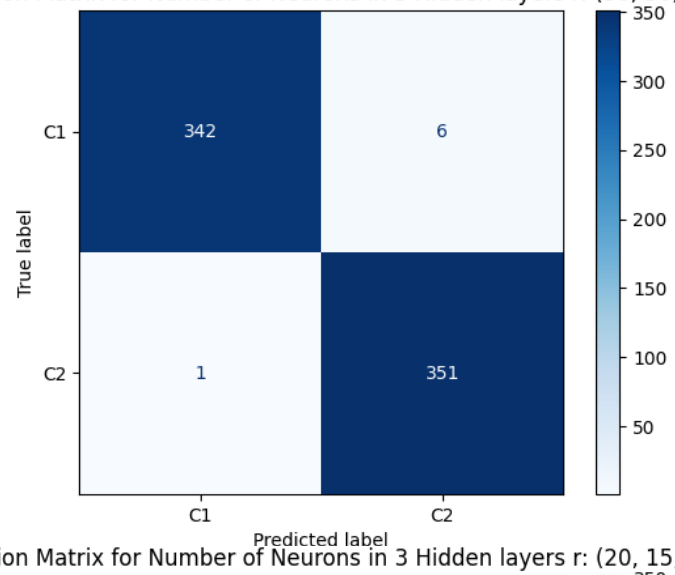
Phase 2: Layer Capacity Fine-tuning

The model was trained using the combination of different neurons number in 3 hidden layers ∈ **{(30,30,30), (20,15,15), (15,15,15), (4, 4, 4)}**

- **Fixed Parameters**: **hidden layers** = 3, η =0.01, α =0.9 , **activation** = ReLU, **epochs** = 150, **batch_size** = 32 , **Dataset Volume:** 7,000 samples , **Normalization**.
- **Objective:** to determine how varying the number of neurons across three hidden layers affects the model's ability to classify interlocking circular patterns.

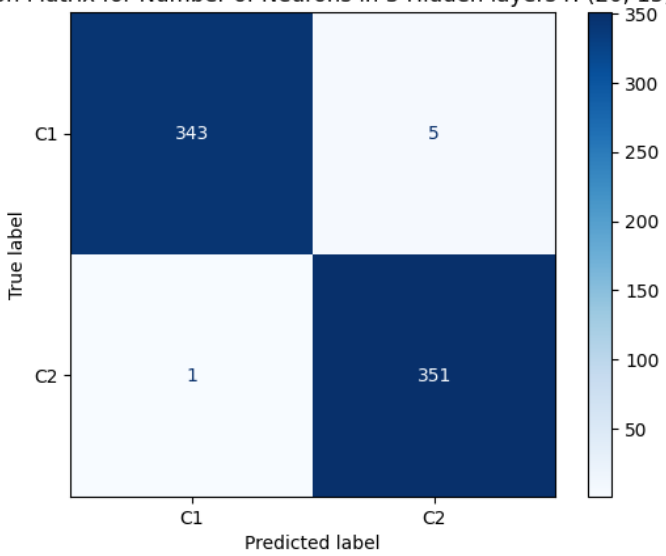


Confusion Matrix for Number of Neurons in 3 Hidden layers r: (30, 30, 30)



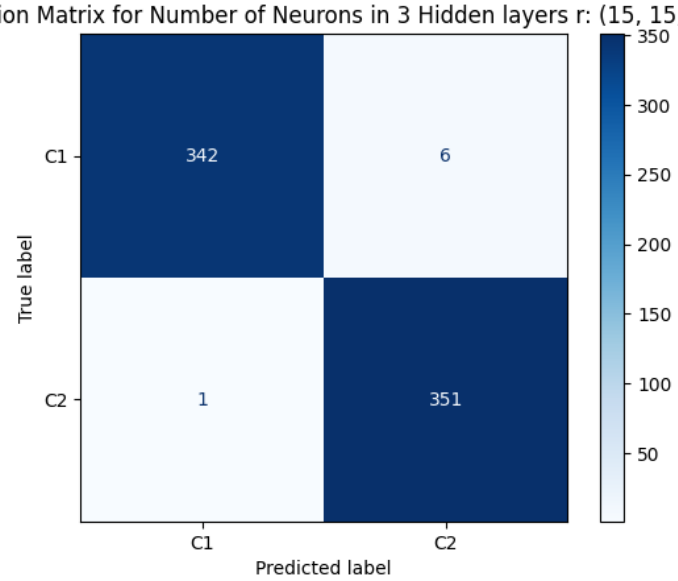

```
classification_report for neourons (30, 30, 30) in 3 hidden layers:
              precision    recall  f1-score   support

           0       1.00      0.98      0.99       348
           1       0.98      1.00      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```

Confusion Matrix for Number of Neurons in 3 Hidden layers r: (20, 15, 15)

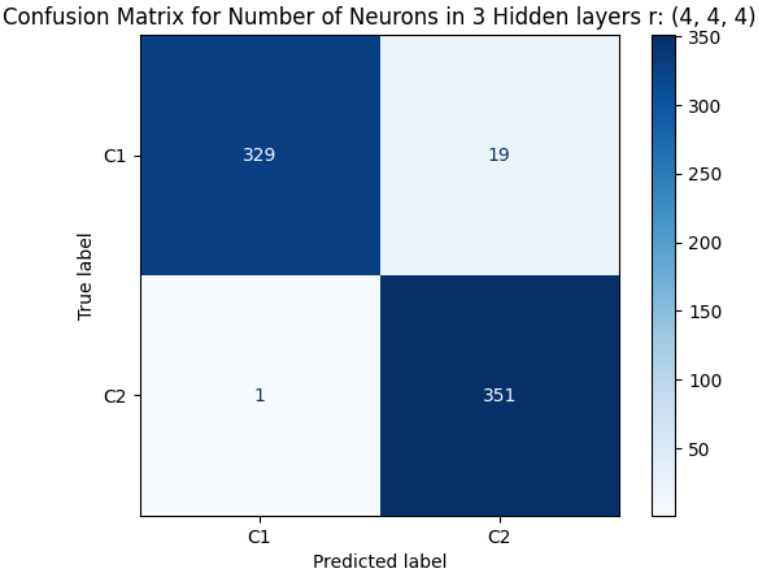



```
classification_report for neourons (20, 15, 15) in 3 hidden layers:
              precision    recall  f1-score   support

           0       1.00      0.99      0.99       348
           1       0.99      1.00      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```

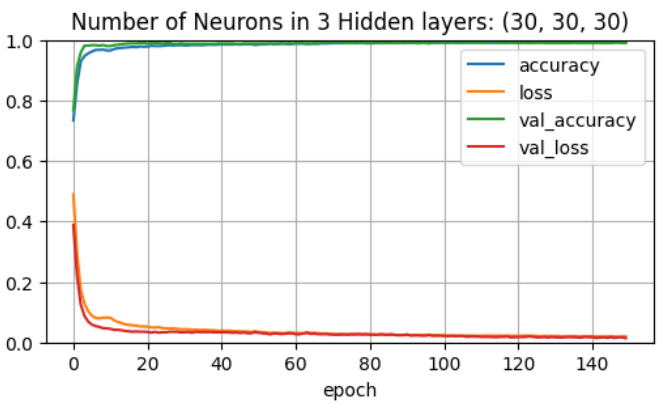
Confusion Matrix for Number of Neurons in 3 Hidden layers r: (15, 15, 15)

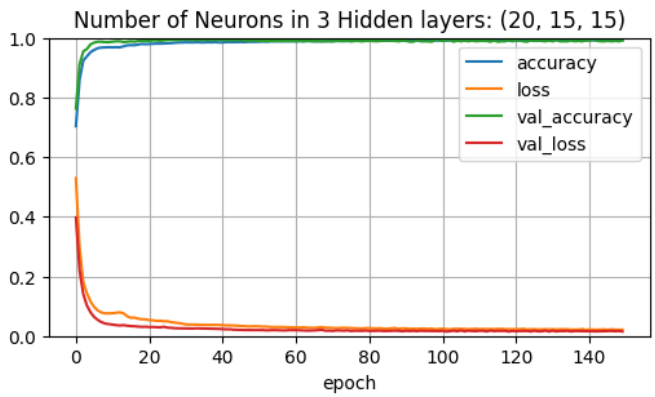


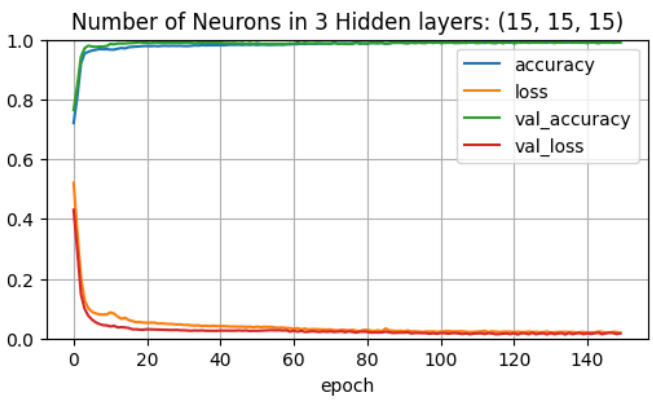

```
classification_report for neourons (15, 15, 15) in 3 hidden layers:
              precision    recall  f1-score   support

           0       1.00      0.98      0.99       348
           1       0.98      1.00      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```

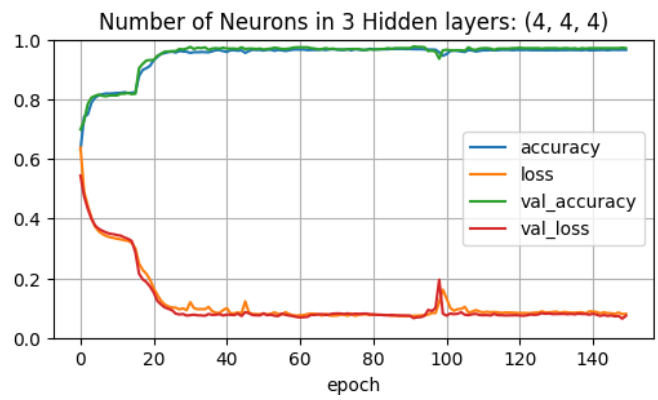



```
classification_report for neourons (4, 4, 4) in 3 hidden layers:
              precision    recall  f1-score   support

           0       1.00      0.95      0.97       348
           1       0.95      1.00      0.97       352

    accuracy                           0.97       700
   macro avg       0.97      0.97      0.97       700
weighted avg       0.97      0.97      0.97       700
```









The **(4, 4, 4)** configuration was too simple and insufficient to fully capture the complex pattern. The training loss curve dropped much slower than the other models, indicating that the network struggled to find the optimal decision boundary. **(accuracy 97%)**

With inceaaring neurons to 15, we observed a significant increse in performance 99% where adding more complexity did not make any big difference in performance .

The models **(15, 15, 15) and (30, 30, 30)** performed almost identically, both reaching **99%** accuracy.

The **(20,15,15)** model provided the **fewest errors (6 errors with accuracy 99%)**.

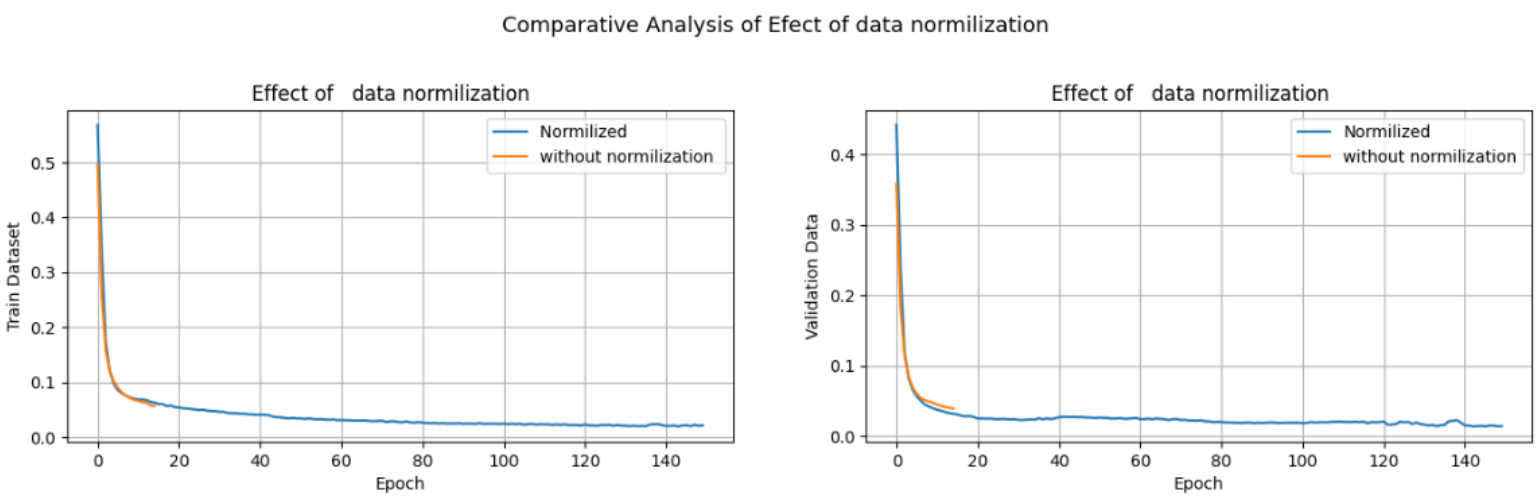
Phase 3: Importance of Data Pre-processing:

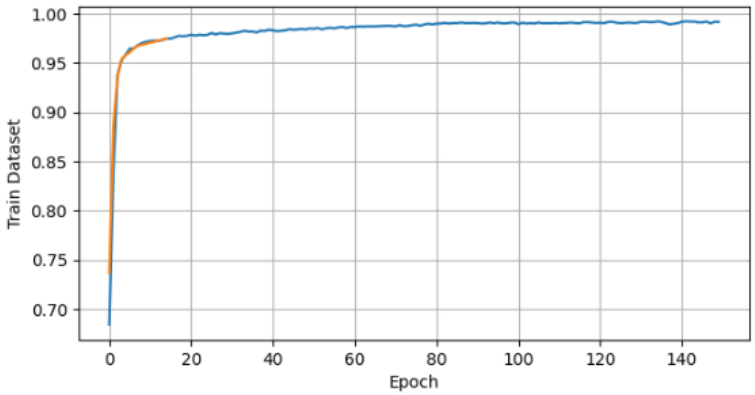
The model was trained with and without input feature normalization to evaluate the effect of data pre-processing on network performance.

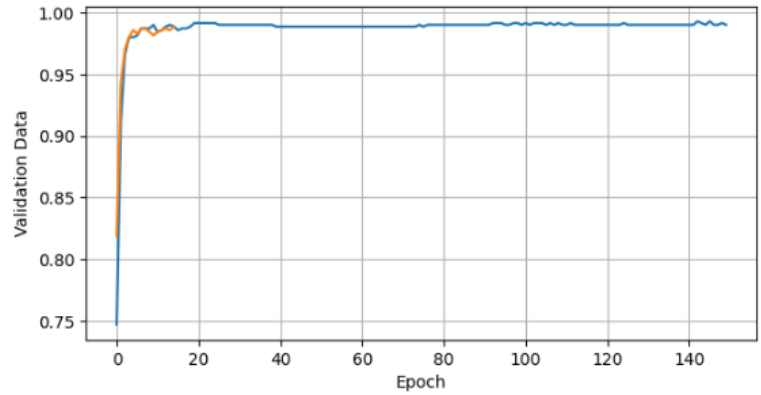
- **Fixed Parameters**: **hidden layers** = 3, η =0.01, α =0.9 , **activation** = ReLU, **epochs** = 150, **batch_size** = 32 , **Dataset Volume:** 7,000 samples , **Number of neurons in each layer** = (20,15,15).

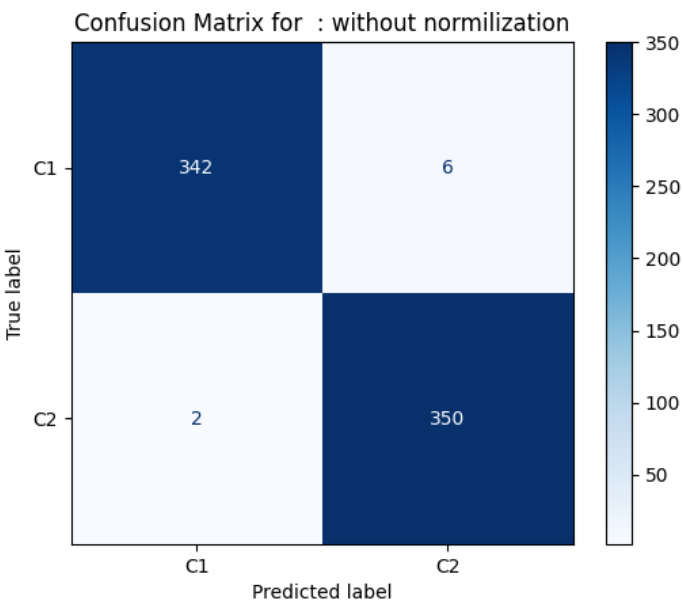
- **Objective:** The goal of this experiment is to observe how normalization influences training stability, convergence speed, and classification accuracy of the neural network.

.

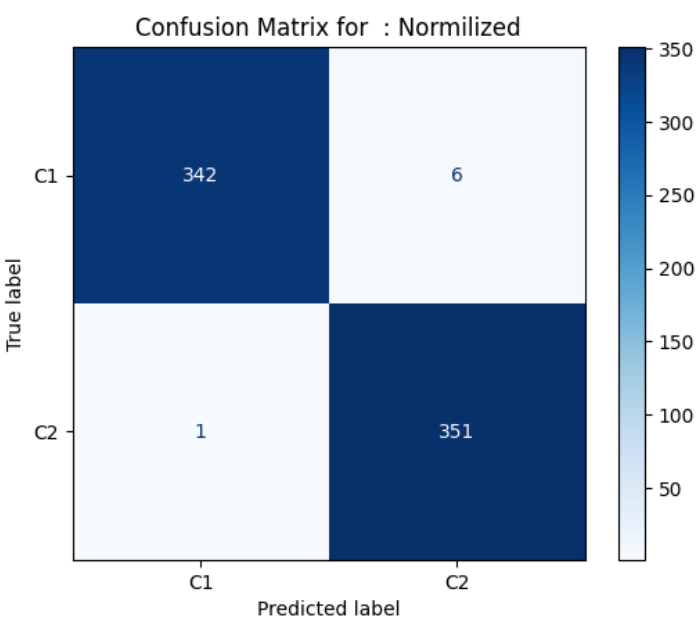




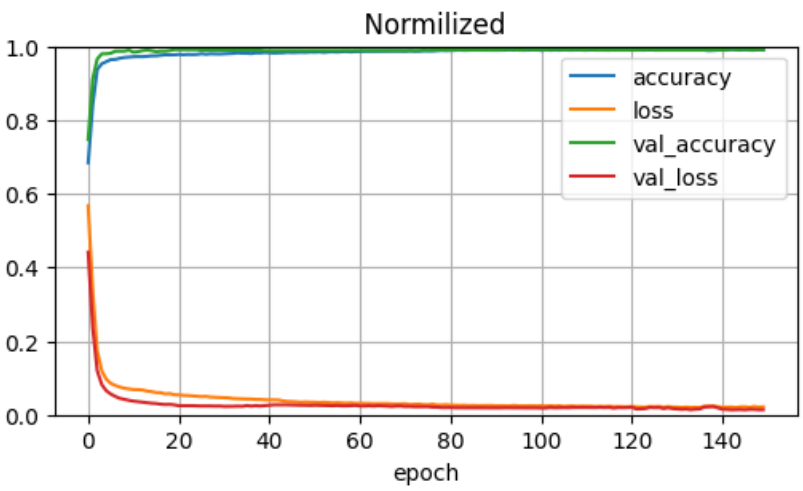


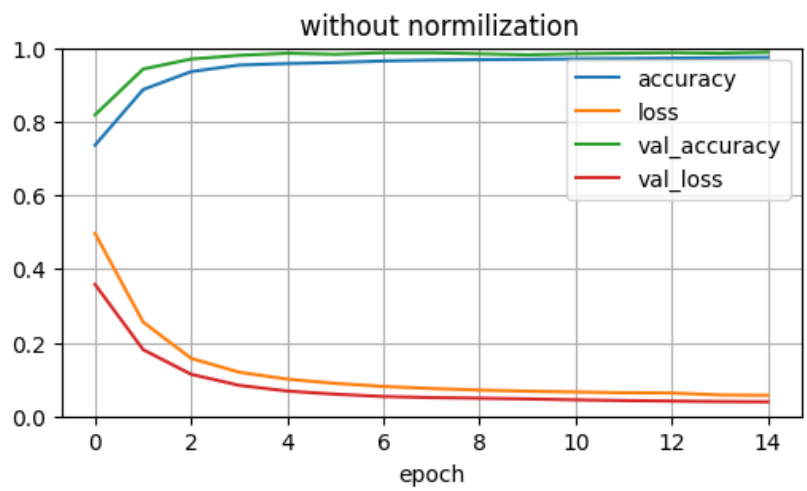



```
classification_report for without normilization  :
              precision    recall  f1-score   support

           0       0.99      0.98      0.99       348
           1       0.98      0.99      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```




```
classification_report for Normilized  :
              precision    recall  f1-score   support

           0       1.00      0.98      0.99       348
           1       0.98      1.00      0.99       352

    accuracy                           0.99       700
   macro avg       0.99      0.99      0.99       700
weighted avg       0.99      0.99      0.99       700
```





While the confusion matrices show a slight **difference** of **only 1 misclassification** between the **normalized (7 errors)** and **non-normalized (8 errors)** models, their training approaches are fundamentally different. **Non-normalized** models at **epoch 14**, suffered from "**exploding gradients**," meaning the numbers became so huge the computer couldn't handle them anymore, causing the whole training to crash and turn into NaN (Not a Number). So, we can't really trust this model because its accuracy was just a result of stochastic luck (a lucky guess before it crashed). In contrast, the **normalized model** was a steady learner. It stayed **stable** and calm, finishing the entire 150-epoch run without any glitches.

Phase 4: Final Model Integration & Testing:

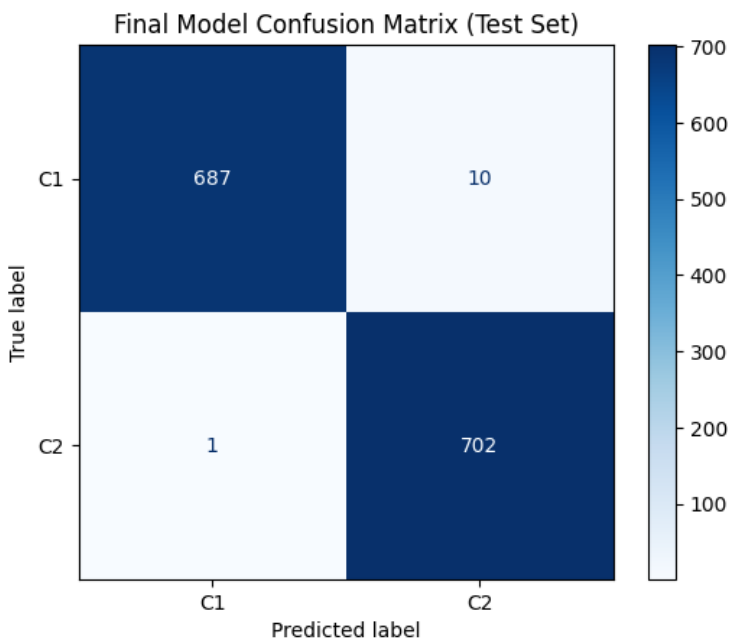
The absolute best combination of hyperparameters was used to train the final model. This model was then evaluated on the completely unseen two Test Set (normal test data, the critical area) to report the final accuracies and generate the confusion matrixes.

Experiment 1 :

evaluate the model on the completely unseen normal Test Set.

- **Fixed Parameters: hidden layers** = 3, η =0.01, α =0.9 , **activation** = ReLU, **epochs** = 150, **batch_size** = 32 , **Dataset Volume:** 7,000 samples , **Number of neurons in each layer** = (20,15,15) **, Normalization**.

- **Objective:**
  To measure the final performance of the model on unseen normal data and observe its classification accuracy.
- **Final Outcome:** Accuracy = 99.21%

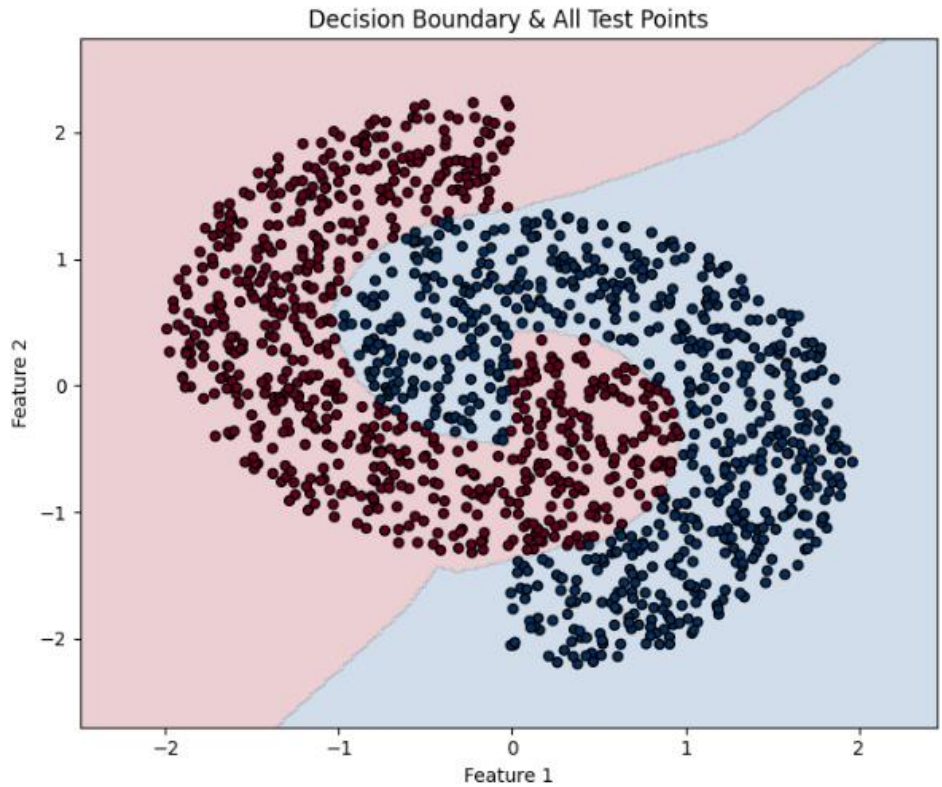


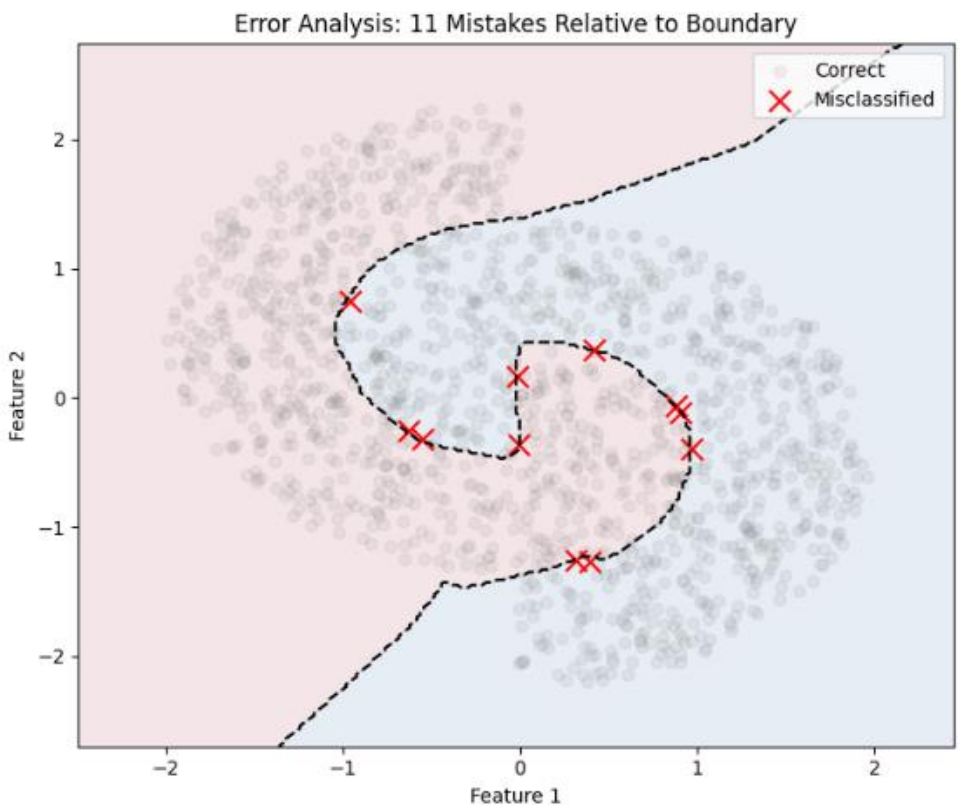

```
Final Classification Report:
              precision    recall  f1-score   support

     Class C1       1.00      0.99      0.99       697
     Class C2       0.99      1.00      0.99       703

    accuracy                           0.99      1400
   macro avg       0.99      0.99      0.99      1400
weighted avg       0.99      0.99      0.99      1400
```

Model: "sequential_325"

Layer (type)	Output Shape	Param #
dense_1300 (Dense)	(None, 20)	60
dense_1301 (Dense)	(None, 15)	315
dense_1302 (Dense)	(None, 15)	240
dense_1303 (Dense)	(None, 1)	16

Total params: 1,264 (4.94 KB)
Trainable params: 631 (2.46 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 633 (2.48 KB)


Decision Boundary & All Test Points
Feature 2
Feature 1
2
1
0
−1
−2
−2
−1
0
1
2

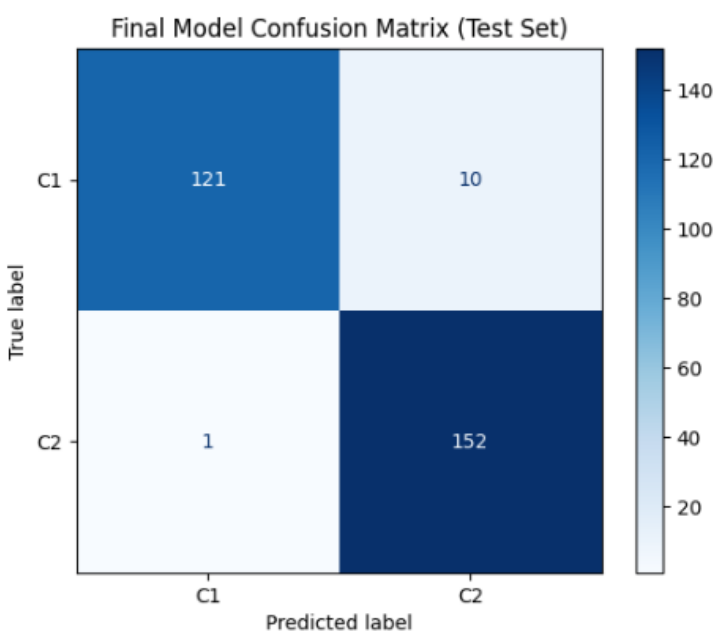

Error Analysis: 11 Mistakes Relative to Boundary
Correct
Misclassified
Feature 2
Feature 1
2
1
0
−1
−2
−2
−1
0
1
2

The final model, utilizing the optimized 3-layer architecture, achieved an exceptional **99.21% accuracy** on the unseen Test Set (1,400 samples). Detailed error analysis reveals that the **11 misclassified** points are located exclusively on the decision boundary where the two classes overlap. This confirms that the model has attained the highest possible theoretical performance for this dataset, demonstrating excellent stability and generalization without any signs of overfitting.

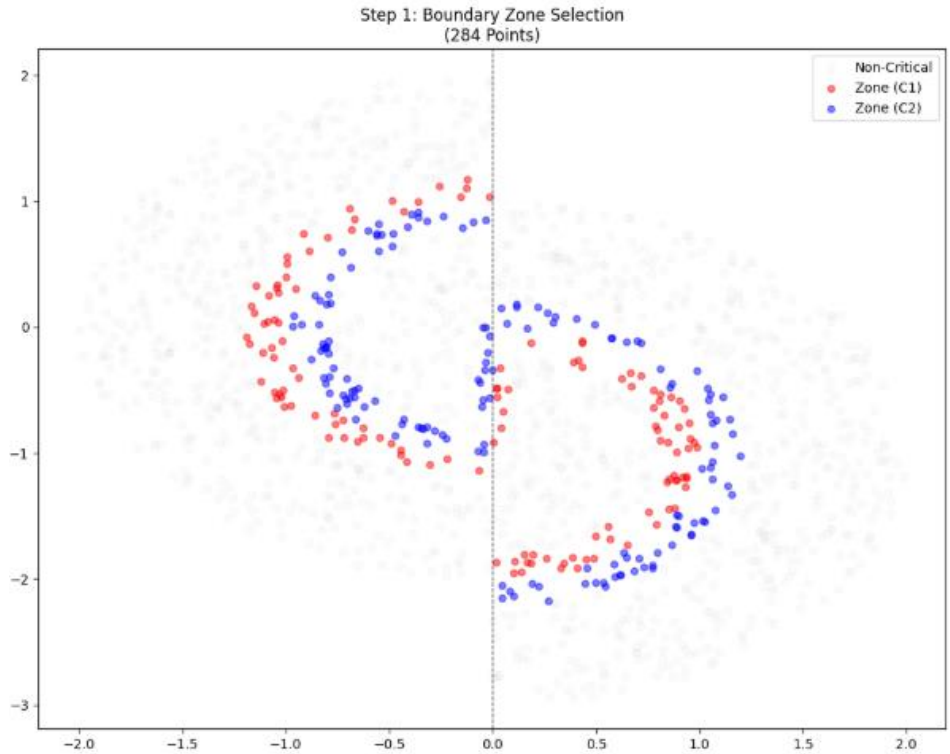
Experiment 2 :

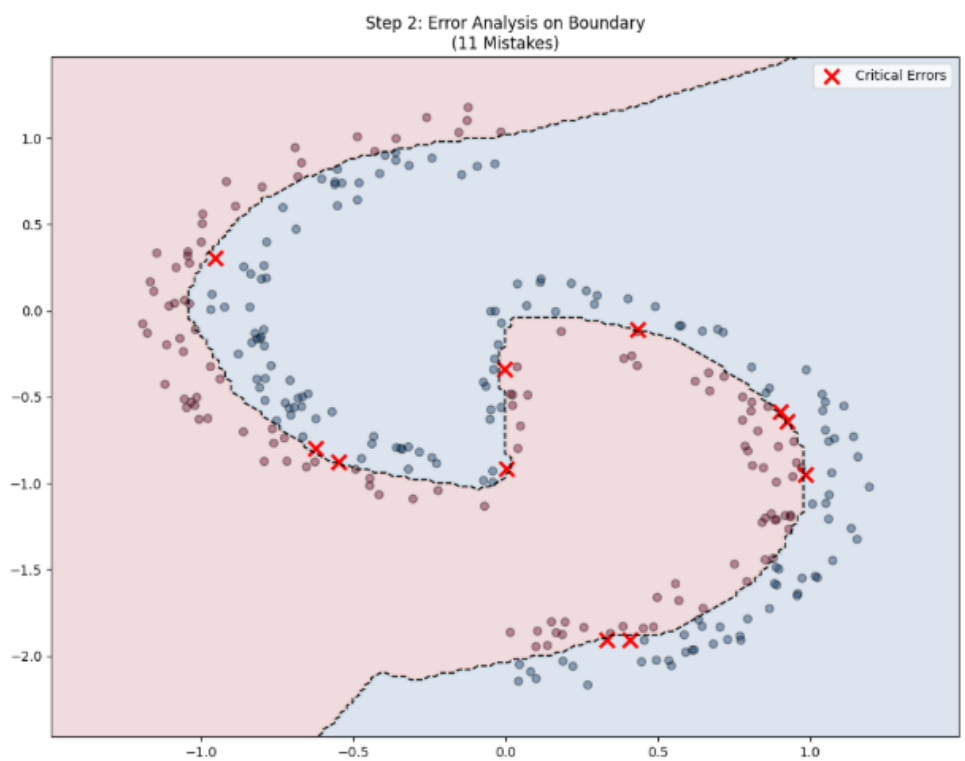
evaluate the model on just completely unseen critical area of Test Set.

- **Fixed Parameters: hidden layers** = 3, $\boldsymbol{\eta}$ =0.01, $\boldsymbol{\alpha}$ =0.9 , **activation** = ReLU, **epochs** = 150, **batch_size** = 32 , **Dataset Volume:** 7,000 samples , **Number of neurons in each layer** = (20,15,15) **, Normalization**.
- **Final Model Accuracy on Boundary Zone: 96.13%**




```
Final Classification Report:
              precision    recall  f1-score   support

     Class C1       0.99      0.92      0.96       131
     Class C2       0.94      0.99      0.97       153

    accuracy                           0.96       284
   macro avg       0.97      0.96      0.96       284
weighted avg       0.96      0.96      0.96       284
```


Step 1: Boundary Zone Selection
(284 Points)
Non-Critical
Zone (C1)
Zone (C2)


Step 2: Error Analysis on Boundary
(11 Mistakes)
Critical Errors

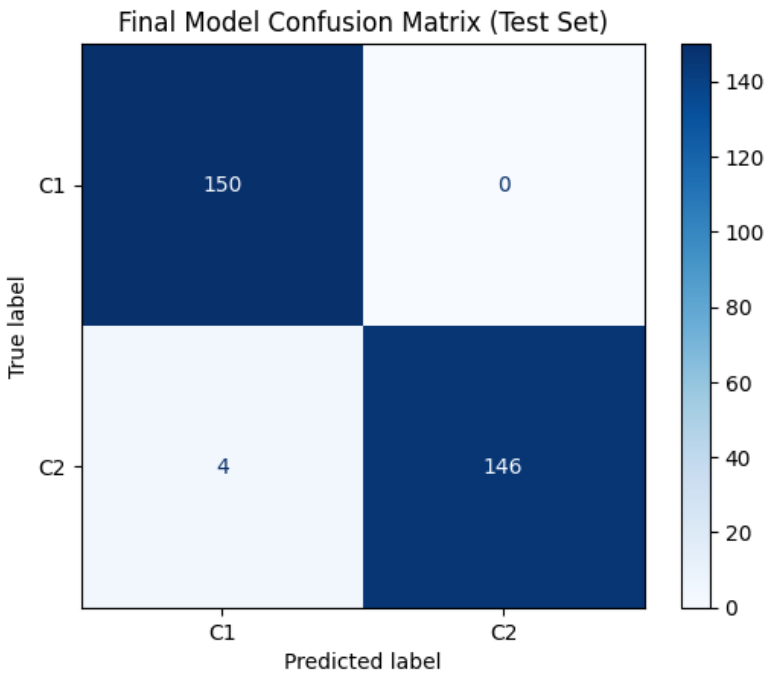
Phase 5: Scalability Study :

Evaluating the model's performance on larger or smaller datasets (3.5k, 15k).

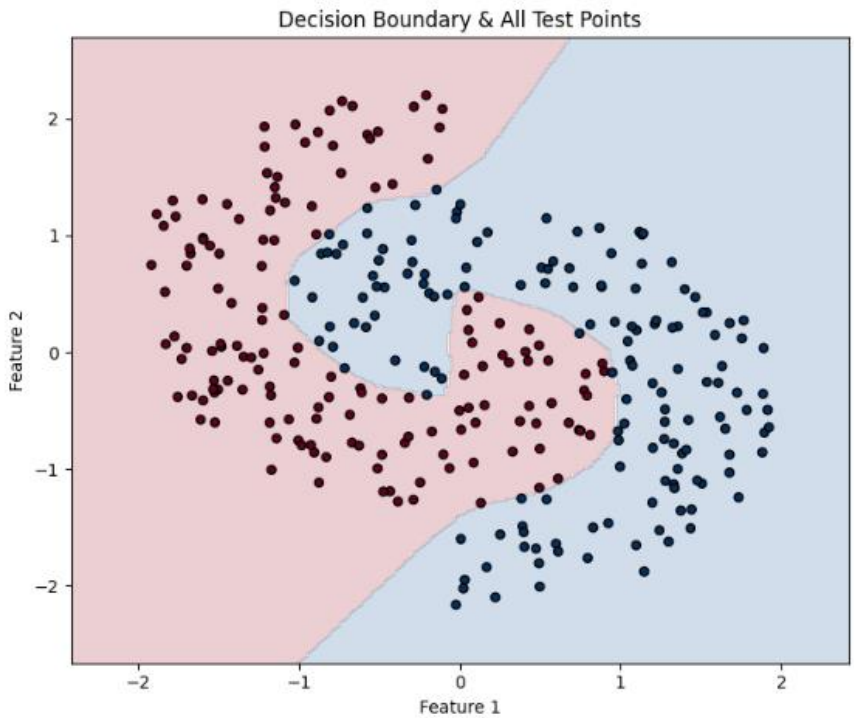
Experiment 1 :

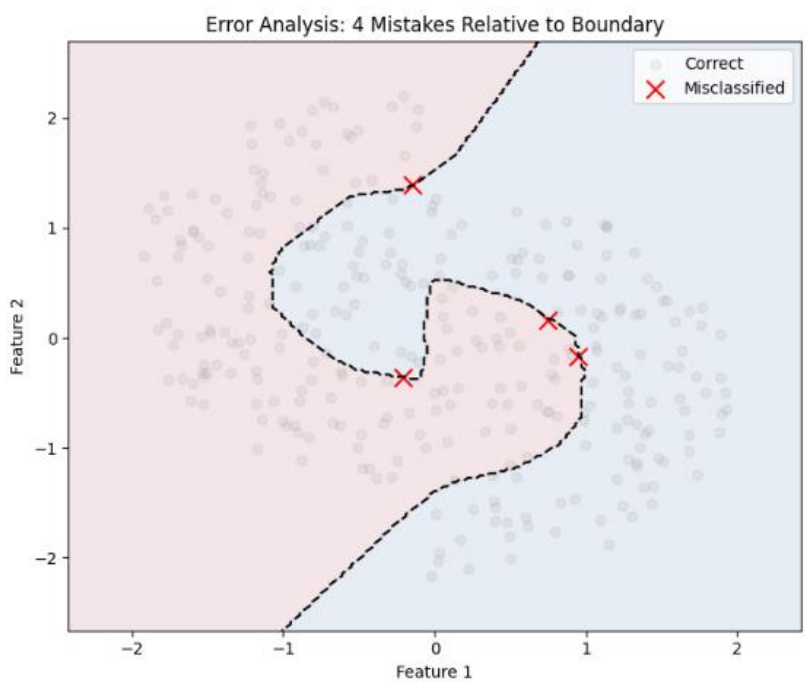
Evaluating the model's performance on smaller **Dataset Volume:** 3,500 samples.

- **Fixed Parameters: hidden layers** = 3, η =0.01, α =0.9 , **activation** = ReLU, **epochs** = 150, **batch_size** = 32 , **Number of neurons in each layer** = (20,15,15) **, Normalization**.
- **Final Model Accuracy on Test data: 98.67%**




```
Final Classification Report:
              precision    recall  f1-score   support

     Class C1       0.97      1.00      0.99       150
     Class C2       1.00      0.97      0.99       150

    accuracy                           0.99       300
   macro avg       0.99      0.99      0.99       300
weighted avg       0.99      0.99      0.99       300
```



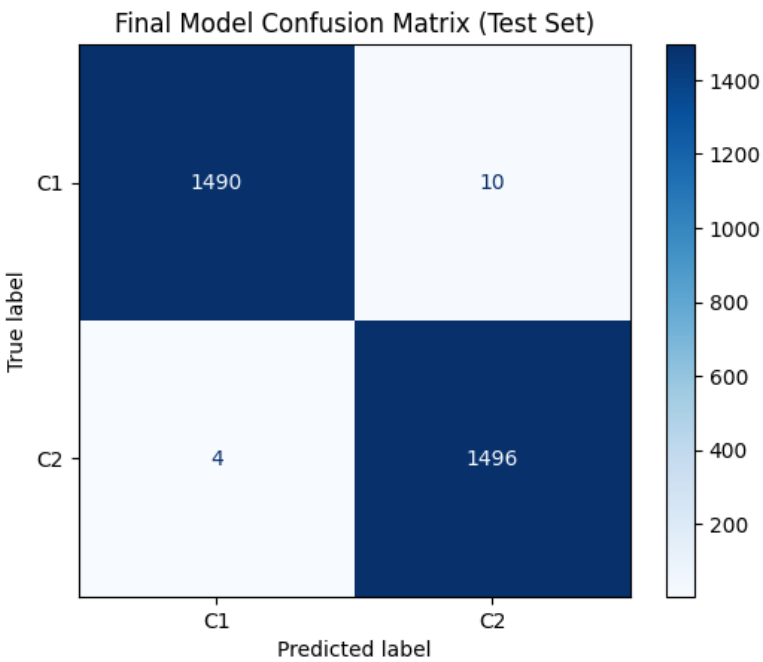


Using the (20, 15, 15) architecture on a dataset of **3,500 samples**, the model achieved a **test accuracy of 98.67%,** with **only 4 misclassified points** out of 300 test samples. Error analysis shows that all 4 errors are located directly on the complex decision boundary.

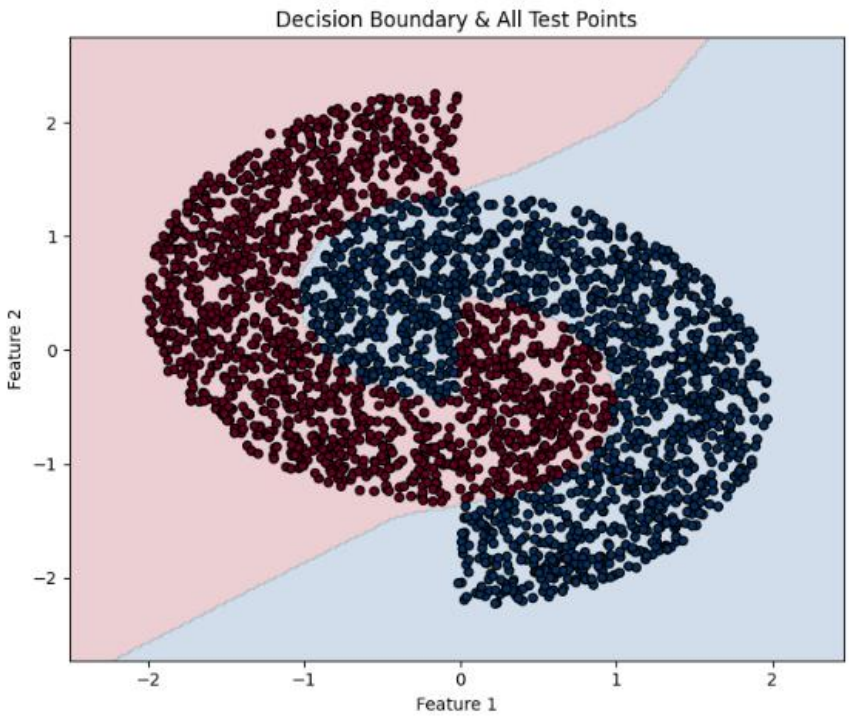
Experiment 2 :

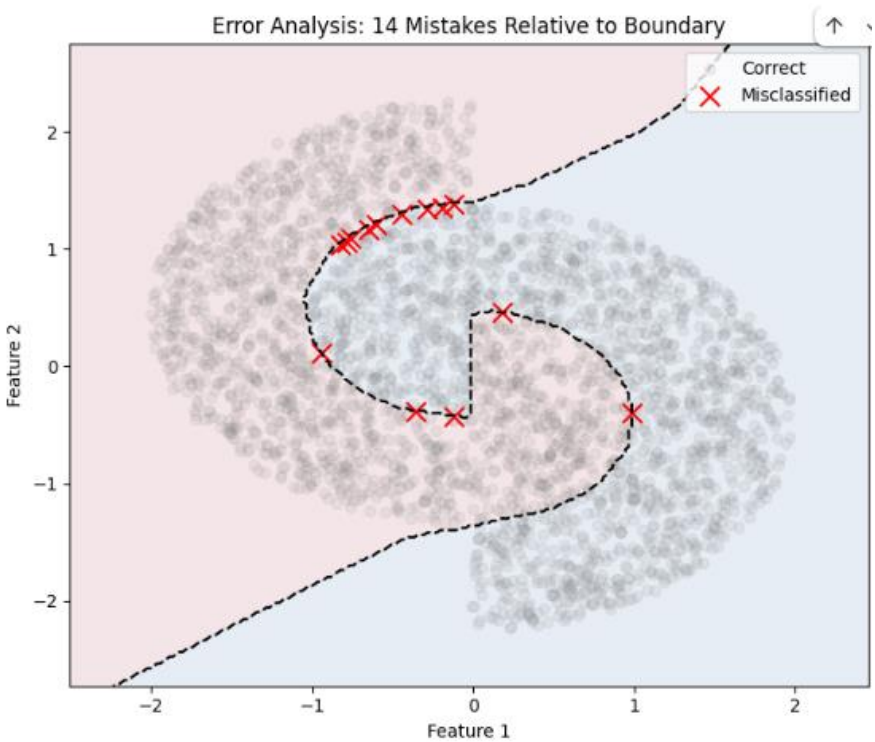
Evaluating the model's performance on larger **dataset Volume:**15,000 samples.

- **Fixed Parameters: hidden layers** = 3, η =0.01, α =0.9 , **activation** = ReLU, **epochs** = 150, **batch_size** = 32 , **Number of neurons in each layer** = (20,15,15) **, Normalization**.
- **Final Model Accuracy on Test data : 99.42 %**




```
Final Classification Report:
              precision    recall  f1-score   support

    Class C1       1.00      0.99      1.00      1500
    Class C2       0.99      1.00      1.00      1500

    accuracy                           1.00      3000
   macro avg       1.00      1.00      1.00      3000
weighted avg       1.00      1.00      1.00      3000
```





The scalability study shows that the optimized (20, 15, 15) architecture generalizes very well. When the dataset size increased from 3,500 to 15,000 samples, the model accuracy went up from 98.67% to 99.42%. In the large dataset experiment (15,000 samples), there were only 14 misclassified points out of 3,000 test samples. Error analysis shows that these mistakes happen only very close to the decision boundary.

Conclusion

This project successfully demonstrated the design, optimization, and scaling of a Multi-Layer Perceptron (MLP) for classifying complex, non-linear interlocking geometries. Through a five-phase approach, we transitioned from a basic underperforming model to a highly robust and scalable architecture (20, 15, 15).

Key Takeaways from the Optimization Journey:

- **Architectural Evolution:** I saw that too few neurons (like 4) caused underfitting, while adding more than 10–15 neurons didn't improve performance much. The final 3-layer architecture had enough capacity to handle the complex, non-convex decision boundaries without overfitting.
- **The Power of Pre-processing:** Phase 3 showed that Normalization is essential for neural networks. It helps keep numbers stable, prevents gradients from exploding, and allows the model to learn faster during training.
- **Hyperparameter Tuning:** By fine-tuning the Learning Rate (0.01) and Momentum (0.9), we identified the "sweet spot" for optimization. This combination allowed the SGD optimizer to navigate the complex loss surface efficiently, reaching a stable global minimum.

Final Performance and Scalability:

- **Boundary Precision**: The model consistently reached about 99% accuracy across all experiments. Importantly, all misclassified points were located very close to the decision boundary. This shows that the errors are due to the overlap and complexity of the classes rather than any problem with the network itself.
- **Robustness at Scale:** The scalability study (Phase 5) was the ultimate validation. Increasing the dataset volume from 3,500 to 15,000 samples actually improved the accuracy to 99.42%. This demonstrates that the model thrives on higher data density and is ready for large-scale real-world applications.

Final Verdict:

The optimized MLP (20, 15, 15) is a highly generalized and stable classifier. It perfectly balances structural efficiency with predictive power, proving that even with high geometric complexity, a well-tuned and normalized neural network can reach near-theoretical performance limits.

Strengths and Limitations of the MLP Model

Based on the results of this project, the strengths and limitations of the Multi-Layer Perceptron (MLP) model were identified as follows:

Advantages

- **Non-linearity & Complex Mapping:** This is the most significant advantage observed in this project. By using activation functions like ReLU and Sigmoid, the MLP successfully modeled the non-convex, interlocking boundaries that are impossible to separate with linear classifiers.
- **High Scalability:** As demonstrated in Phase 5, the MLP architecture thrives on larger datasets. When the data volume increased to 15,000 samples, the model accuracy actually improved, proving its ability to extract finer patterns from higher-density data.
- **Versatility:** The MLP showed it could handle different data scales (from 3,500 to 15,000 samples) without needing a change in the core architecture.

Disadvantages

- **Sensitivity to Data Scaling (Critical):** This project highlighted that MLPs are highly sensitive to the range of input features. In Phase 3, we observed that without proper Normalization, the model could suffer from vanishing gradients or unstable training, making pre-processing a mandatory step.
- **Prone to Overfitting:** During Phase 2, we saw that adding too many neurons (e.g., moving to 20+ neurons) provides diminishing returns and increases the risk of the model "memorizing" the training data instead of generalizing.
- **Computational Complexity:** While effective, training an MLP on large datasets (like the 15k sample set) is more computationally expensive than simpler models. This requires efficient use of batch sizes and epochs to balance training time and accuracy.